

+12,000 kg	12,000
-2,500 kg	9,500
-1,000 kg	8,500
+6,000 kg	14,500
-3,000 kg	11,500
-0,750 kg	10,750
= 10,500 kg	10,500

Como montar um sistema de estoque com ASP.NET Core e Next.js

Guia passo a passo do projeto MioDolce: criação da solução, models, migrations, services, requests, controllers, configuração, frontend e a conversa entre os dois lados.

Sumário

Os itens são clicáveis. No leitor de PDF, o painel de marcadores mostra a mesma estrutura.

você escreve código digitado por você

gerado criado por um comando (dotnet, npx)

gerado + editado o comando cria e você ajusta

Parte 1 Visão geral

1. O que vamos construir
2. Arquitetura em camadas
3. Mapa: onde fica cada coisa

Parte 2 Preparando o ambiente

4. Ferramentas
5. Banco de dados com Docker

Parte 3 Backend em ASP.NET Core

6. Criando a solução e os projetos
7. Configurações compartilhadas
8. Domain: os models
9. Application: requests, responses, validação e services
10. Infrastructure: DbContext e mapeamento
11. Migrations
12. Api: Program.cs, controllers e filtros
13. Configuração: appsettings, conexão e segredos
14. Testes
15. Receita: um recurso novo do zero

Parte 4 Frontend em Next.js

16. Criando o app Next.js
17. Rotas, layout e estilo
18. Acesso à API
19. Lendo dados: Server Components
20. Gravando dados: Server Actions e formulários

Parte 5 Next.js conversando com o ASP.NET

21. O caminho de uma leitura e de uma gravação
22. Contratos, erros e CORS

Parte 6 Rodando e evoluindo

23. Rodar, testar e resolver problemas
24. Glossário

Visão geral

O que o sistema faz, como as peças se encaixam e onde cada arquivo mora. Leia esta parte antes de digitar qualquer comando.

1 O que vamos construir

O MioDolce controla o estoque do almoxarifado de uma confeitaria. Ele tem cinco recursos:

Recurso	O que faz	Regra principal
Categorias	Agrupam produtos (Chocolates, Embalagens...)	Nome único; só exclui se estiver vazia
Produtos	SKU, unidade, custo, saldo e estoque mínimo	O saldo nunca é editado direto
Movimentações	Entrada, saída e ajuste de inventário	Saída não pode passar do saldo; histórico imutável
Fornecedores	Nome, CNPJ, e-mail, telefone	CNPJ único, guardado só com dígitos
Painel	Valor em estoque, alertas, fluxo de 14 dias	Somente leitura

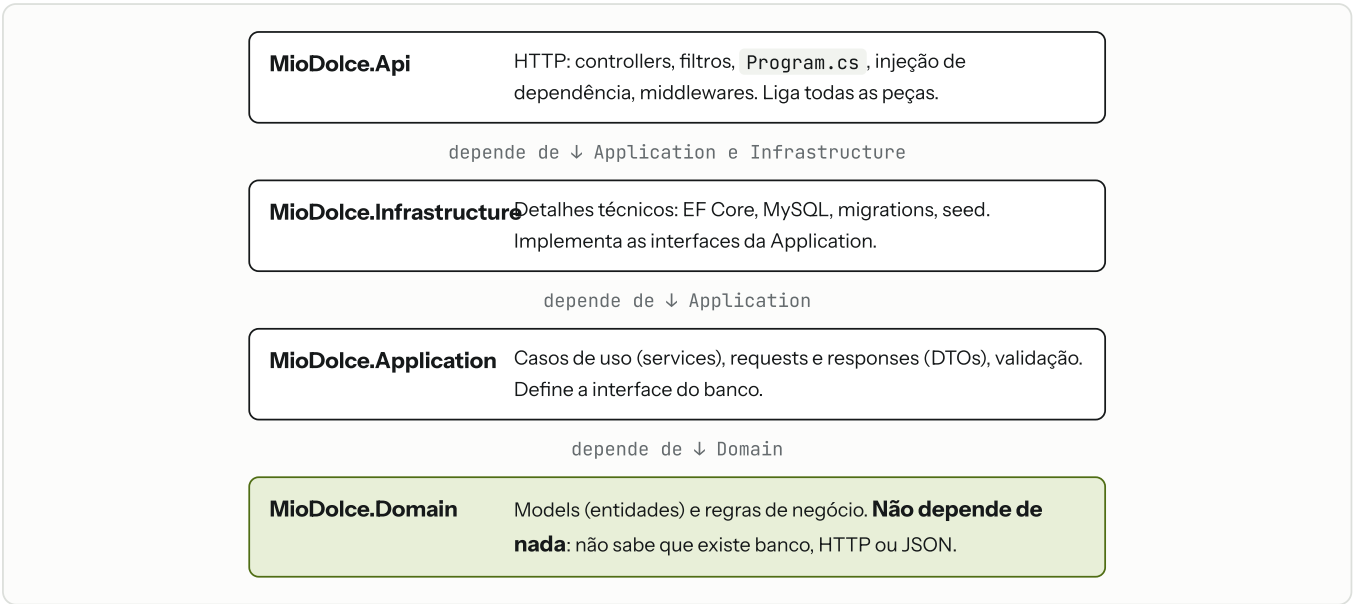
O sistema tem três programas rodando ao mesmo tempo:



O navegador **nunca** chama a API diretamente: quem chama é o servidor do Next.js. A Parte 5 explica por quê e como.

2 Arquitetura em camadas

O backend é dividido em quatro projetos (Clean Architecture enxuta). Cada um tem uma responsabilidade e só pode depender dos projetos "de dentro":



Por que separar assim?

Porque cada tipo de mudança fica num lugar só. Trocar MySQL por PostgreSQL mexe só na Infrastructure. Trocar Controllers por Minimal APIs mexe só na Api (este projeto fez exatamente isso). Mudar uma regra de estoque mexe só no Domain. E o compilador impede atalhos: o Domain não consegue nem enxergar o EF Core.

O caminho de uma requisição atravessa as camadas de fora para dentro e volta:

1	Controller	recebe o HTTP, o ASP.NET converte o JSON em um request (DTO)
2	Filtro de validação	valida o request; se inválido, devolve 400 e para aqui
3	Service	carrega o que precisa do banco e chama o domínio
4	Model (Domain)	aplica a regra de negócio e muda o próprio estado, ou devolve um erro
5	DbContext (EF Core)	transforma as mudanças em SQL e grava numa transação
6	Controller	converte o resultado em response (DTO) e status HTTP (200, 201, 404...)

3 Mapa: onde fica cada coisa

A coluna "Quem cria" mostra o que você digita e o que as ferramentas geram. Tudo marcado **você escreve** é programação feita por uma pessoa.

Raiz do repositório

Caminho	O que é	Quem cria
<code>docker-compose.yml</code>	Sobe o MySQL 8.4 em container, porta 3307	você escreve
<code>README.md</code> , <code>docs/</code>	Instruções e guias de estudo	você escreve
<code>.gitignore</code> , <code>.vscode/extensions.json</code>	Arquivos ignorados pelo git; extensões recomendadas	você escreve

backend/ (solução .NET)

Caminho	O que é	Quem cria
<code>MioDolce.slnx</code>	Solução: lista os projetos	gerado <code>dotnet new sln</code> <code>/ dotnet sln add</code>
<code>global.json</code>	Fixa a versão do SDK e o executor de testes	gerado + editado
<code>Directory.Build.props</code>	Configurações comuns a todos os <code>.csproj</code>	você escreve
<code>Directory.Packages.props</code>	Versões de todos os pacotes NuGet (num lugar só)	você escreve
<code>.editorconfig</code>	Regras de estilo aplicadas no build	você escreve
<code>dotnet-tools.json</code>	Ferramentas locais (dotnet-ef)	gerado <code>dotnet tool install</code>
<code>src/*/*.csproj</code>	Definição de cada projeto e suas referências	gerado + editado

MioDolce.Domain: models e regras

Caminho	O que é	Quem cria
<code>Common/Entity.cs</code>	Base com <code>Id</code> (Guid v7)	você escreve
<code>Common/AuditableEntity.cs</code>	Base com datas de auditoria e token de concorrência	você escreve

Caminho	O que é	Quem cria
<code>Common/Result.cs</code> , <code>Error.cs</code>	Result pattern: erros de negócio como valores	você escreve
<code>Common/Guard.cs</code>	Cláusulas de guarda (invariantes)	você escreve
<code>Categories/Category.cs</code>	Model de categoria	você escreve
<code>Products/Product.cs</code>	Model de produto com a regra de saldo	você escreve
<code>Products/UnitOfMeasure.cs</code> , <code>StockStatus.cs</code>	Enums	você escreve
<code>Stock/StockMovement.cs</code> , <code>MovementType.cs</code>	Model do livro-razão e seu enum	você escreve
<code>Suppliers/Supplier.cs</code>	Model de fornecedor	você escreve
<code>*/*Errors.cs</code>	Catálogo de erros de cada model	você escreve

MioDolce.Application: casos de uso

Caminho	O que é	Quem cria
<code>Abstractions/IApplicationDbContext.cs</code>	Interface do banco que os services usam	você escreve
<code>Common/Paging.cs</code> , <code>PagingValidation.cs</code>	Resposta paginada e suas regras	você escreve
<code>Common/DbContextExtensions.cs</code> , <code>CommonErrors.cs</code>	Salvar tratando conflito de concorrência	você escreve
<code>Features/<Recurso>/*Contracts.cs</code>	Requests e responses (DTOs)	você escreve
<code>Features/<Recurso>/*Validator(s).cs</code>	Regras de validação (FluentValidation)	você escreve
<code>Features/<Recurso>/*Service.cs</code>	Services: consultas e casos de uso	você escreve
<code>Features/Products/ProductMappings.cs</code> , <code>...QueryExtensions.cs</code>	Entidade para DTO; filtros e ordenação	você escreve
<code>DependencyInjection.cs</code>	Registra services e validadores	você escreve

MioDolce.Infrastructure: banco de dados

Caminho	O que é	Quem cria
<code>Persistence/AppDbContext.cs</code>	O DbContext: um DbSet por tabela	você escreve
<code>Persistence/Configurations/*.cs</code>	Mapeamento model para tabela (Fluent API)	você escreve
<code>Persistence/Migrations/*_Nome.cs</code>	Migration: Up() e Down()	gerado dotnet ef migrations add
<code>Persistence/Migrations/*.Designer.cs</code> , <code>AppDbContextModelSnapshot.cs</code>	Metadados e "foto" do modelo	gerado nunca edite
<code>Persistence/Interceptors/AuditEntityInterceptor.cs</code>	Preenche datas e token antes de salvar	você escreve
<code>Persistence/Seed/InventorySeeder.cs</code>	Dados de exemplo	você escreve
<code>Persistence/DatabaseOptions.cs</code>	Configuração tipada da seção "Database"	você escreve
<code>DependencyInjection.cs</code>	Registra DbContext, MySQL, health check	você escreve

MioDolce.Api: HTTP

Caminho	O que é	Quem cria
<code>Program.cs</code>	Ponto de entrada: serviços e pipeline	gerado + editado

Caminho	O que é	Quem cria
<code>DependencyInjection.cs</code>	Controllers, JSON, ProblemDetails, OpenAPI, CORS	você escreve
<code>Controllers/ApiControllerBase.cs</code>	Base dos controllers, converte erro em HTTP	você escreve
<code>Controllers/*Controller.cs</code>	Controllers: um por recurso	você escreve
<code>Filters/FluentValidationActionFilter.cs</code>	Roda os validadores antes das actions	você escreve
<code>Infrastructure/GlobalExceptionHandler.cs</code>	Erro inesperado vira 500 padronizado	você escreve
<code>Infrastructure/FrontendOptions.cs</code>	Configuração tipada do CORS	você escreve
<code>Extensions/DatabaseExtensions.cs</code>	Aplica migrations ao subir (dev)	você escreve
<code>appsettings.json</code> , <code>appsettings.Development.json</code>	Configuração: conexão, logs, CORS	gerado + editado
<code>Properties/launchSettings.json</code>	Porta 5080 e ambiente ao rodar local	gerado + editado
<code>MioDolce.Api.http</code>	Requisições prontas para testar	você escreve

backend/tests

Caminho	O que é	Quem cria
<code>MioDolce.Domain.Tests/**</code>	Testes de unidade das regras (sem banco)	você escreve
<code>MioDolce.Api.IntegrationTests/Infrastructure/ApiFactory.cs</code>	Sobe a API + MySQL em container para os testes	você escreve
<code>MioDolce.Api.IntegrationTests/*ControllerTests.cs</code>	Testes HTTP de ponta a ponta	você escreve

frontend/ (Next.js)

Caminho	O que é	Quem cria
<code>package.json</code> , <code>tsconfig.json</code> , <code>next.config.ts</code> , <code>eslint.config.mjs</code> , <code>postcss.config.mjs</code>	Configuração do projeto	gerado <code>create-next-app</code>
<code>.env.example</code> / <code>.env.local</code>	URL da API (<code>API_URL</code>)	você escreve
<code>src/app/layout.tsx</code> , <code>globals.css</code>	Moldura de todas as páginas; tokens de cor	gerado + editado
<code>src/app/**/page.tsx</code>	Uma página por rota (pasta = URL)	você escreve
<code>src/app/**/loading.tsx</code> , <code>error.tsx</code> , <code>not-found.tsx</code>	Carregando, erro e 404	você escreve
<code>src/features/<assunto>/types.ts</code>	Tipos TypeScript espelhando os DTOs do C#	você escreve
<code>src/features/<assunto>/api.ts</code>	Uma função por endpoint da API	você escreve
<code>src/features/<assunto>/actions.ts</code>	Server Actions (gravação)	você escreve
<code>src/features/<assunto>/components/</code>	Formulários, tabelas, gráficos do assunto	você escreve
<code>src/components/ui/</code> , <code>layout/</code>	Botões, campos, cabeçalho: UI genérica	você escreve
<code>src/lib/api/client.ts</code>	O único ponto que faz <code>fetch</code> na API	você escreve
<code>src/lib/*.ts</code>	Ambiente, formatação, formulários, URL	você escreve

Preparando o ambiente

Instale uma vez e esqueça. Os comandos deste guia funcionam no PowerShell, no Git Bash e no terminal do VS Code.

4 Ferramentas

Ferramenta	Para quê	Como instalar / conferir
.NET 10 SDK	compilar e rodar o backend, criar projetos, migrations	winget install Microsoft.DotNet.SDK.10 conferir: dotnet --list-sdks deve listar 10.0.x
Node.js 20+	rodar o Next.js (o projeto usa Node 24)	winget install OpenJS.NodeJS.LTS conferir: node -v
Docker Desktop	MySQL em container e testes de integração	docker.com, depois docker info
VS Code + extensões	editor	C# Dev Kit, REST Client, Tailwind CSS IntelliSense, ESLint, EditorConfig

SDK x runtime

Ter o *runtime* do .NET não basta: `dotnet new`, `dotnet build` e `dotnet ef` exigem o **SDK**. Se `dotnet --list-sdks` não mostrar nada, o SDK não está instalado (era o caso desta máquina no início do projeto).

5 Banco de dados com Docker

Em vez de instalar o MySQL na máquina, ele roda num container descrito por um arquivo. Crie `docker-compose.yml` na raiz do repositório **você escreve** :

MySQL 8.4 para desenvolvimento

docker-compose.yml

```
# MySQL para desenvolvimento local.
# Subir: docker compose up -d
# Parar: docker compose down          (mantém os dados)
# Zerar: docker compose down -v      (apaga o volume → banco recriado e re-populado na próxima execução da API)
services:
  mysql:
    image: mysql:8.4 # 8.4 é a versão LTS atual do MySQL
    container_name: miodolce-mysql
    restart: unless-stopped
    environment:
      MYSQL_DATABASE: miodolce
      MYSQL_USER: miodolce
      MYSQL_PASSWORD: miodolce_dev
      MYSQL_ROOT_PASSWORD: root_dev
      TZ: UTC
    ports:
      # 3307 no host para não colidir com um MySQL instalado localmente (que costuma usar a 3306).
      - "${MYSQL_HOST_PORT:-3307}:3306"
    volumes:
      - mysql-data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-proot_dev"]
      interval: 5s
      timeout: 3s
      retries: 20

volumes:
  mysql-data:
```

```
$ docker compose up -d      # sobe em segundo plano
$ docker compose ps         # deve mostrar "healthy"
$ docker compose down       # para (os dados continuam no volume)
$ docker compose down -v    # para e APAGA os dados
```

Por que a porta 3307?

Dentro do container o MySQL escuta na 3306 (padrão). No seu computador ele aparece na 3307, porque a 3306 costuma estar ocupada por um MySQL instalado localmente. A connection string da API usa `Port=3307`.

Backend em ASP.NET Core

Da solução vazia até a API com controllers, banco, migrations e testes. A ordem dos capítulos é a ordem em que você deve construir.

6 Criando a solução e os projetos

Uma **solução** (`.slnx`) agrupa **projetos** (`.csproj`). Cada camada da arquitetura é um projeto separado. Tudo abaixo é **gerado** por comandos; você só digita os comandos.

6.1 Solução e projetos

```
$ mkdir backend && cd backend
$ dotnet new sln -n MioDolce --format slnx
# camadas (classlib = biblioteca; web = aplicação ASP.NET Core vazia)
$ dotnet new classlib -n MioDolce.Domain -o src/MioDolce.Domain -f net10.0
$ dotnet new classlib -n MioDolce.Application -o src/MioDolce.Application -f net10.0
$ dotnet new classlib -n MioDolce.Infrastructure -o src/MioDolce.Infrastructure -f net10.0
$ dotnet new web -n MioDolce.Api -o src/MioDolce.Api -f net10.0
# testes
$ dotnet new classlib -n MioDolce.Domain.Tests -o tests/MioDolce.Domain.Tests -f net10.0
$ dotnet new classlib -n MioDolce.Api.IntegrationTests -o tests/MioDolce.Api.IntegrationTests -f net10.0
# registra todos na solução e apaga o Class1.cs que o template cria
$ dotnet sln add src/MioDolce.Domain src/MioDolce.Application src/MioDolce.Infrastructure src/MioDolce.Api
$ dotnet sln add tests/MioDolce.Domain.Tests tests/MioDolce.Api.IntegrationTests
# fixa a versão do SDK usada pelo repositório
$ dotnet new globaljson --sdk-version 10.0.401 --roll-forward latestFeature
```

web ou webapi?

`dotnet new web` cria um projeto vazio: você escreve tudo e entende cada linha. A alternativa `dotnet new webapi --use-controllers` já vem com controllers, OpenAPI e um `WeatherForecastController` de exemplo para apagar. As duas chegam ao mesmo lugar.

6.2 Referências entre projetos (a regra de dependência)

```
$ dotnet add src/MioDolce.Application reference src/MioDolce.Domain
$ dotnet add src/MioDolce.Infrastructure reference src/MioDolce.Application
$ dotnet add src/MioDolce.Api reference src/MioDolce.Application src/MioDolce.Infrastructure
$ dotnet add tests/MioDolce.Domain.Tests reference src/MioDolce.Domain
$ dotnet add tests/MioDolce.Api.IntegrationTests reference src/MioDolce.Api
```

O resultado são linhas `<ProjectReference>` nos `.csproj`. Veja como ficaram o da Application e o da Api:

backend/src/MioDolce.Application/MioDolce.Application.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <!-- Application: casos de uso. Conhece o Domain e define abstrações que a Infrastructure implementa. -->
  <ItemGroup>
    <ProjectReference Include="..\MioDolce.Domain\MioDolce.Domain.csproj" />
  </ItemGroup>

  <ItemGroup>
    <!-- Só a API base do EF Core (DbSet, IQueryable async). O provider do MySQL fica na Infrastructure. -->
    <PackageReference Include="Microsoft.EntityFrameworkCore" />
    <PackageReference Include="FluentValidation.DependencyInjectionExtensions" />
  </ItemGroup>
</Project>
```

backend/src/MioDolce.Api/MioDolce.Api.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <!-- Api: camada de apresentação HTTP e "composition root" (onde a injeção de dependência é montada). -->
  <PropertyGroup>
    <UserSecretsId>miodolce-api-6b1f3c1e</UserSecretsId>
  </PropertyGroup>

  <ItemGroup>
    <ProjectReference Include="..\MioDolce.Application\MioDolce.Application.csproj" />
    <ProjectReference Include="..\MioDolce.Infrastructure\MioDolce.Infrastructure.csproj" />
  </ItemGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.OpenApi" />
    <PackageReference Include="Scalar.AspNetCore" />
    <!-- Necessário para "dotnet ef migrations add" usar este projeto como startup. -->
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design">
      <PrivateAssets>all</PrivateAssets>
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
    </PackageReference>
  </ItemGroup>
</Project>
```

6.3 Pacotes NuGet

Pacote	Projeto	Para quê
Microsoft.EntityFrameworkCore	Application	DbSet , consultas assíncronas
FluentValidation.DependencyInjectionExtensions	Application	validadores + registro automático no DI
MySQL.EntityFrameworkCore	Infrastructure	provider do MySQL para o EF Core (oficial da Oracle)
Microsoft.EntityFrameworkCore.Relational	Infrastructure	recursos de banco relacional (migrations, índices)
Microsoft.Extensions.Diagnostics.HealthChecks.EntityFrameworkCore	Infrastructure	health check do banco
Microsoft.Extensions.Options.ConfigurationExtensions	Infrastructure	BindConfiguration do Options pattern
Microsoft.AspNetCore.OpenApi , Scalar.AspNetCore	Api	documento OpenAPI e a interface visual /scalar
Microsoft.EntityFrameworkCore.Design	Api	necessário para dotnet ef migrations
xunit.v3	testes	framework de testes
Microsoft.AspNetCore.Mvc.Testing , Testcontainers.MySql	testes de integração	API em memória + MySQL descartável

Para adicionar um pacote: `dotnet add src/MioDolce.Application package FluentValidation.DependencyInjectionExtensions`. Como o projeto usa *Central Package Management* (próximo capítulo), a **versão** vai para `Directory.Packages.props` e o `.csproj` recebe só o nome.

6.4 A ferramenta dotnet-ef

```
$ dotnet new tool-manifest # cria dotnet-tools.json
$ dotnet tool install dotnet-ef --version 10.0.12 # registra a versão no manifesto
$ dotnet tool restore # em outra máquina: instala o que o manifesto pede
```

Ferramenta **local** (no manifesto) garante que todo mundo do projeto usa a mesma versão do `dotnet ef`.

7 Configurações compartilhadas

Três arquivos na pasta `backend/` valem para **todos** os projetos, sem repetir nada nos `.csproj`. Todos são **você escreve**.

7.1 Directory.Build.props

O MSBuild importa este arquivo automaticamente em cada projeto abaixo da pasta. Por isso os `.csproj` do projeto são quase vazios.

backend/Directory.Build.props

```
<Project>
  <!--
    Directory.Build.props é importado automaticamente por TODOS os .csproj abaixo desta pasta.
    Centralizar aqui evita repetir configurações e garante que todos os projetos sigam as mesmas regras.
  -->
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <LangVersion>latest</LangVersion>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>

    <!-- Qualidade: avisos viram erros e os analisadores recomendados da Microsoft ficam ligados. -->
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <AnalysisLevel>latest-recommended</AnalysisLevel>
    <EnforceCodeStyleInBuild>true</EnforceCodeStyleInBuild>

    <!-- Builds reproduzíveis e determinísticos. -->
    <Deterministic>true</Deterministic>
    <InvariantGlobalization>>false</InvariantGlobalization>
  </PropertyGroup>
</Project>
```

- `Nullable` : o compilador avisa quando algo pode ser `null` . Combinado com `TreatWarningsAsErrors` , esquecer uma checagem de nulo impede o build.
- `AnalysisLevel latest-recommended` : liga os analisadores de qualidade da Microsoft (performance, segurança, boas práticas).

7.2 Directory.Packages.props (Central Package Management)

backend/Directory.Packages.props

```
<Project>
  <!--
    Central Package Management (CPM): as VERSÕES dos pacotes NuGet ficam só aqui.
    Nos .csproj usamos <PackageReference Include="..." /> sem versão.
    Assim todos os projetos usam a mesma versão e a atualização é feita em um único lugar.
  -->
  <PropertyGroup>
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
  </PropertyGroup>

  <ItemGroup Label="ASP.NET Core">
    <PackageVersion Include="Microsoft.AspNetCore.OpenApi" Version="10.0.12" />
    <PackageVersion Include="Scalar.AspNetCore" Version="2.17.10" />
  </ItemGroup>

  <ItemGroup Label="Entity Framework Core + MySQL">
    <PackageVersion Include="Microsoft.EntityFrameworkCore" Version="10.0.12" />
    <PackageVersion Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.12" />
    <PackageVersion Include="Microsoft.EntityFrameworkCore.Design" Version="10.0.12" />
    <PackageVersion Include="MySQL.EntityFrameworkCore" Version="10.0.9" />
    <PackageVersion Include="Microsoft.Extensions.Diagnostics.HealthChecks.EntityFrameworkCore" Version="10.0.12" />
    <PackageVersion Include="Microsoft.Extensions.Options.ConfigurationExtensions" Version="10.0.12" />
  </ItemGroup>

  <ItemGroup Label="Validação">
    <PackageVersion Include="FluentValidation.DependencyInjectionExtensions" Version="12.1.1" />
  </ItemGroup>

  <ItemGroup Label="Testes">
    <PackageVersion Include="xunit.v3" Version="4.0.1" />
    <PackageVersion Include="Microsoft.AspNetCore.Mvc.Testing" Version="10.0.12" />
    <PackageVersion Include="Testcontainers.MySql" Version="4.15.0" />
  </ItemGroup>
</Project>
```

7.3 global.json e .editorconfig

```
backend/global.json

{
  "sdk": {
    "version": "10.0.401",
    "rollForward": "latestFeature"
  },
  "test": {
    "runner": "Microsoft.Testing.Platform"
  }
}
```

A seção "test" ativa a **Microsoft Testing Platform**, o executor de testes novo do .NET 10 que o xUnit v3 usa. O `.editorconfig` define o estilo (namespaces de arquivo, campos privados com `_`, chaves obrigatórias) e é aplicado no build por causa de `EnforceCodeStyleInBuild`. Cada regra desligada ali tem um comentário explicando o motivo.

8 Domain: os models

8.1 "Model" tem dois significados

Em tutoriais de ASP.NET é comum uma pasta `Models/` com classes que servem de tabela e de JSON ao mesmo tempo. Aqui os dois papéis são separados:

	Model de domínio (entidade)	Models da API (request / response)
Exemplo	<code>Supplier</code>	<code>SupplierRequest</code> , <code>SupplierResponse</code>
Mora em	<code>MioDolce.Domain</code>	<code>MioDolce.Application/Features/...</code>
Representa	a tabela e as regras de negócio	o JSON que entra e que sai
Formato	classe com setters privados e métodos	<code>record</code> só com dados

Se fosse a mesma classe, qualquer coluna nova do banco vazaria na API, e o cliente poderia enviar campos que não deveria (como o saldo do produto).

8.2 Classes base

```
backend/src/MioDolce.Domain/Common/Entity.cs

namespace MioDolce.Domain.Common;

/// <summary>
/// Classe base de toda entidade: algo que tem identidade própria.
/// Usamos Guid versão 7 (ordenado pelo tempo), que evita a fragmentação de índice
/// que um Guid aleatório (v4) causaria no MySQL.
/// </summary>
public abstract class Entity
{
    public Guid Id { get; private init; } = Guid.CreateVersion7();
}
```

```
backend/src/MioDolce.Domain/Common/AuditableEntity.cs

namespace MioDolce.Domain.Common;

/// <summary>
/// Entidade com trilha de auditoria e controle de concorrência.
/// Os valores são preenchidos automaticamente por um interceptor do EF Core
/// (veja Infrastructure/Persistence/Interceptors), então o domínio não precisa se preocupar com isso.
/// </summary>
public abstract class AuditableEntity : Entity
{
    public DateTime CreatedAtUtc { get; private set; }
```

```

public DateTime? UpdatedAtUtc { get; private set; }

/// <summary>
/// Token de concorrência otimista: muda a cada UPDATE.
/// Se duas requisições editarem o mesmo registro ao mesmo tempo, a segunda falha
/// em vez de sobrescrever silenciosamente a primeira.
/// </summary>
public Guid ConcurrencyStamp { get; private set; }
}

```

8.3 Erros como valores: Result e Error

Erro **esperado** (estoque insuficiente, CNPJ repetido) não é exceção: é um valor que o método devolve. Quem chama é obrigado a olhar `IsSuccess`. Exceção fica para o inesperado (banco fora do ar, bug).

backend/src/MioDolce.Domain/Common/Error.cs

```

namespace MioDolce.Domain.Common;

/// <summary>Categoria do erro. A API traduz cada tipo para um status HTTP.</summary>
public enum ErrorType
{
    Failure,
    Validation,
    NotFound,
    Conflict,
    BusinessRule,
}

/// <summary>
/// Erro esperado de negócio (ex.: "estoque insuficiente").
/// Erros esperados são VALORES retornados, não exceções: exceções ficam para o inesperado.
/// </summary>
public sealed record Error(string Code, string Description, ErrorType Type)
{
    public static readonly Error None = new(string.Empty, string.Empty, ErrorType.Failure);

    public static Error NotFound(string code, string description) => new(code, description, ErrorType.NotFound);

    public static Error Conflict(string code, string description) => new(code, description, ErrorType.Conflict);

    public static Error BusinessRule(string code, string description) => new(code, description, ErrorType.BusinessRule);

    public static Error Validation(string code, string description) => new(code, description, ErrorType.Validation);
}

```

A classe Result (a versão genérica Result<T> vem logo abaixo no arquivo)

backend/src/MioDolce.Domain/Common/Result.cs · trecho

```

namespace MioDolce.Domain.Common;

/// <summary>
/// Result pattern: um método que pode falhar por regra de negócio retorna Result
/// em vez de lançar exceção. Quem chama é obrigado a olhar IsSuccess/IsFailure.
/// </summary>
public class Result
{
    protected Result(bool isSuccess, Error error)
    {
        if (isSuccess && error != Error.None)
        {
            throw new ArgumentException("Um resultado de sucesso não pode conter erro.", nameof(error));
        }

        if (!isSuccess && error == Error.None)
        {
            throw new ArgumentException("Um resultado de falha precisa de um erro.", nameof(error));
        }

        IsSuccess = isSuccess;
        Error = error;
    }

    public bool IsSuccess { get; }
}

```

```

public bool IsFailure => !IsSuccess;

public Error Error { get; }

public static Result Success() => new(true, Error.None);

public static Result Failure(Error error) => new(false, error);

public static Result<TValue> Success<TValue>(TValue value) => new(value, true, Error.None);

public static Result<TValue> Failure<TValue>(Error error) => new(default, false, error);

public static implicit operator Result(Error error) => Failure(error);

```

8.4 Um model completo: Supplier

Receita de um model de domínio **você escreve** :

1. herde de `AuditableEntity` (ganha `Id`, datas e token de concorrência);
2. declare os limites como **constantes** (serão usados pelo banco e pelo validador);
3. propriedades com `private set` : só a própria classe altera seus dados;
4. um **construtor privado vazio** (o EF Core precisa dele para ler do banco);
5. um **factory method** `Create(...)` : a única forma de criar um objeto válido;
6. métodos com nome de negócio para alterar o estado (`Update` , `Archive` , `RegisterMovement`).

backend/src/MioDolce.Domain/Suppliers/Supplier.cs

```

using MioDolce.Domain.Common;

namespace MioDolce.Domain.Suppliers;

/// <summary>
/// PASSO 1 do guia docs/06: o MODEL (entidade).
///
/// Em ASP.NET, "model" costuma significar duas coisas diferentes:
/// 1. o modelo de DOMÍNIO / de dados: esta classe, que vira a tabela "suppliers" no banco;
/// 2. os modelos de ENTRADA e SAÍDA da API (DTOs): SupplierRequest / SupplierResponse,
///    em Application/Features/Suppliers/SupplierContracts.cs.
/// Separar os dois evita expor o banco na API e permite mudar um sem quebrar o outro.
/// </summary>
public sealed class Supplier : AuditableEntity
{
    public const int NameMaxLength = 120;
    public const int TaxIdLength = 14; // CNPJ: 14 dígitos
    public const int EmailMaxLength = 254;
    public const int PhoneMaxLength = 20;

    // Exigido pelo EF Core para criar a instância ao ler do banco.
    private Supplier()
    {
    }

    public string Name { get; private set; } = string.Empty;

    /// <summary>CNPJ guardado só com dígitos ("12345678000190"), sem pontuação.</summary>
    public string TaxId { get; private set; } = string.Empty;

    public string? Email { get; private set; }

    public string? Phone { get; private set; }

    public static Supplier Create(string name, string taxId, string? email, string? phone)
    {
        var supplier = new Supplier();
        supplier.Update(name, taxId, email, phone);
        return supplier;
    }

    public void Update(string name, string taxId, string? email, string? phone)
    {
        var digits = NormalizeTaxId(taxId);
        if (digits.Length != TaxIdLength)
        {

```

```

        throw new ArgumentException($"O CNPJ deve ter {TaxIdLength} dígitos.", nameof(taxId));
    }

    Name = Guard.Text(name, NameMaxLength);
    TaxId = digits;
    Email = Guard.OptionalText(email, EmailMaxLength)?.ToLowerInvariant();
    Phone = Guard.OptionalText(phone, PhoneMaxLength);
}

/// <summary>"12.345.678/0001-90" → "12345678000190".</summary>
public static string NormalizeTaxId(string? value) =>
    new((value ?? string.Empty).Where(char.IsAsciiDigit).ToArray());
}

```

Catálogo de erros do model

backend/src/MioDolce.Domain/Suppliers/SupplierErrors.cs

```

using MioDolce.Domain.Common;

namespace MioDolce.Domain.Suppliers;

public static class SupplierErrors
{
    public static readonly Error TaxIdAlreadyExists = Error.Conflict(
        "Supplier.TaxIdAlreadyExists",
        "Já existe um fornecedor com esse CNPJ.");

    public static Error NotFound(Guid id) => Error.NotFound(
        "Supplier.NotFound",
        $"Fornecedor '{id}' não encontrado.");
}

```

8.5 Regra de negócio no model: o saldo do produto

O `Product` é o model mais rico. O saldo (`QuantityOnHand`) só muda por um método, que valida a regra e devolve a movimentação a ser gravada. É impossível "esquecer" a regra em algum lugar do sistema:

Product.RegisterMovement: o único ponto que altera o saldo

backend/src/MioDolce.Domain/Products/Product.cs · trecho

```

/// <summary>
/// Ponto único de alteração de saldo.
/// Para Entrada/Saída, <paramref name="quantity"/> é a quantidade movimentada.
/// Para Ajuste, é o saldo CONTADO fisicamente (o sistema calcula a diferença).
/// </summary>
public Result<StockMovement> RegisterMovement(MovementType type, decimal quantity, string? note, DateTime occurredAtUtc)
{
    if (!IsActive)
    {
        return ProductErrors.Inactive;
    }

    return type switch
    {
        MovementType.Entry => Entry(quantity, note, occurredAtUtc),
        MovementType.Exit => Exit(quantity, note, occurredAtUtc),
        MovementType.Adjustment => AdjustTo(quantity, note, occurredAtUtc),
        _ => throw new ArgumentOutOfRangeException(nameof(type), type, "Tipo de movimentação desconhecido."),
    };
}

private Result<StockMovement> Entry(decimal quantity, string? note, DateTime occurredAtUtc)
{
    if (quantity <= 0)
    {
        return ProductErrors.QuantityMustBePositive;
    }

    return Apply(MovementType.Entry, quantity, note, occurredAtUtc);
}

private Result<StockMovement> Exit(decimal quantity, string? note, DateTime occurredAtUtc)
{
    if (quantity <= 0)
    {
        return ProductErrors.QuantityMustBePositive;
    }
}

```

```

    }

    if (quantity > QuantityOnHand)
    {
        return ProductErrors.InsufficientStock(QuantityOnHand, quantity);
    }

    return Apply(MovementType.Exit, -quantity, note, occurredAtUtc);
}

```

9 Application: requests, responses, validação e services

A Application organiza o código **por recurso**: tudo de fornecedores fica em `Features/Suppliers/`. Cada recurso tem até quatro tipos de arquivo:

Arquivo	Contém
<code>SupplierContracts.cs</code>	requests (entrada) e responses (saída)
<code>SupplierRequestValidator.cs</code>	regras de formato dos requests
<code>SupplierService.cs</code>	os casos de uso: listar, buscar, criar, editar, excluir
<code>ProductMappings.cs</code> (quando necessário)	conversão entidade para response, reaproveitável

9.1 A interface do banco

A Application não conhece o MySQL. Ela declara uma interface com o que precisa, e a Infrastructure implementa (inversão de dependência):

```

backend/src/MioDolce.Application/Abstractions/IApplicationDbContext.cs

using Microsoft.EntityFrameworkCore;
using MioDolce.Domain.Categories;
using MioDolce.Domain.Products;
using MioDolce.Domain.Stock;
using MioDolce.Domain.Suppliers;

namespace MioDolce.Application.Abstractions;

/// <summary>
/// Abstração do banco de dados vista pela Application (Inversão de Dependência – o "D" do SOLID).
/// A Application declara O QUE precisa; a Infrastructure fornece o COMO (AppDbContext + MySQL).
///
/// Por que não um "Repository" para cada entidade? O DbContext do EF Core já implementa
/// os padrões Repository (DbSet) e Unit of Work (SaveChanges). Envolver isso em outra camada
/// costuma só adicionar código sem benefício real em um projeto deste tamanho.
/// </summary>
public interface IApplicationDbContext
{
    DbSet<Category> Categories { get; }

    DbSet<Product> Products { get; }

    DbSet<StockMovement> StockMovements { get; }

    DbSet<Supplier> Suppliers { get; }

    Task<int> SaveChangesAsync(CancellationToken cancellationToken = default);
}

```

9.2 Requests e responses

`record` é o tipo ideal para DTO: imutável, igualdade por valor e declarado numa linha. Os nomes das propriedades viram o JSON em camelCase (`TaxId` vira `taxId`).

```

backend/src/MioDolce.Application/Features/Suppliers/SupplierContracts.cs

namespace MioDolce.Application.Features.Suppliers;

```

```
// PASSO 5 do guia docs/06: os MODELS da API (DTOs).
// O que entra (Request) e o que sai (Response) são contratos públicos, separados da entidade.
// Exemplo da diferença: o Request aceita CNPJ com pontuação ("12.345.678/0001-90");
// a entidade normaliza e o Response devolve só os dígitos. A Response também expõe as datas
// de auditoria, mas NÃO expõe o ConcurrencyStamp, que é detalhe interno.

public sealed record SupplierResponse(
    Guid Id,
    string Name,
    string TaxId,
    string? Email,
    string? Phone,
    DateTime CreatedAtUtc,
    DateTime? UpdatedAtUtc);

public sealed record SupplierRequest(string Name, string TaxId, string? Email, string? Phone);
```

Para listagens com filtros, o request é um record com valores padrão. Cada parâmetro vira um item da query string (?search=choco&page=2):

Request de listagem com filtros e paginação

backend/src/MioDolce.Application/Features/Products/ProductContracts.cs · trecho

```
/// <summary>
/// Parâmetros de query string de GET /api/products.
/// No controller é recebido com [FromQuery]: cada parâmetro do construtor vira um ?parametro=.
/// Valores padrão tornam o parâmetro opcional.
/// </summary>
public sealed record ProductListQuery(
    string? Search = null,
    Guid? CategoryId = null,
    StockStatus? Status = null,
    bool IncludeInactive = false,
    ProductSort Sort = ProductSort.Name,
    int Page = 1,
    int PageSize = Paging.DefaultPageSize) : IPagedQuery;
```

9.3 Validação com FluentValidation

O validador descreve as regras de **formato** do request. Você não registra nem chama o validador: o DI o encontra sozinho e um filtro o executa antes da action (capítulo 12).

backend/src/MioDolce.Application/Features/Suppliers/SupplierRequestValidator.cs

```
using FluentValidation;
using MioDolce.Domain.Suppliers;

namespace MioDolce.Application.Features.Suppliers;

/// <summary>
/// PASSO 6 do guia docs/06: validação de entrada.
/// Não precisa registrar à mão: AddValidatorsFromAssembly (Application/DependencyInjection.cs)
/// encontra este validador, e o FluentValidationActionFilter o executa antes da action.
/// </summary>
internal sealed class SupplierRequestValidator : AbstractValidator<SupplierRequest>
{
    public SupplierRequestValidator()
    {
        RuleFor(x => x.Name)
            .NotEmpty()
            .MaxLength(Supplier.NameMaxLength)
            .WithName("Nome");

        // Aceita "12.345.678/0001-90" ou "12345678000190": conta só os dígitos.
        RuleFor(x => x.TaxId)
            .NotEmpty()
            .Must(taxId => Supplier.NormalizeTaxId(taxId).Length == Supplier.TaxIdLength)
            .WithMessage("O CNPJ deve ter 14 dígitos.")
            .WithName("CNPJ");

        RuleFor(x => x.Email)
            .EmailAddress()
            .MaxLength(Supplier.EmailMaxLength)
            .When(x => !string.IsNullOrEmpty(x.Email))
            .WithName("E-mail");
    }
}
```

```

        RuleFor(x => x.Phone)
            .MaxLength(Supplier.PhoneMaxLength)
            .WithName("Telefone");
    }
}

```

Validação x regra de negócio

"Nome é obrigatório" e "CNPJ tem 14 dígitos" são validação: dá para decidir só olhando o request (resposta 400). "Não pode sair mais que o saldo" e "CNPJ já cadastrado" dependem do banco: ficam no domínio ou no service (respostas 422 e 409).

9.4 O service

O service é onde o trabalho acontece: consulta o banco, chama o domínio e salva. Os quatro movimentos do EF Core aparecem aqui:

Operação	Código	SQL gerado no SaveChanges
ler	<code>db.Suppliers.AsNoTracking().Where(...).Select(ToResponse)</code>	SELECT só das colunas usadas
criar	<code>db.Suppliers.Add(supplier)</code>	INSERT
alterar	<code>FindAsync(id)</code> e depois <code>supplier.Update(...)</code>	UPDATE só do que mudou
excluir	<code>db.Suppliers.Remove(supplier)</code>	DELETE

backend/src/MioDolce.Application/Features/Suppliers/SupplierService.cs

```

using System.Linq.Expressions;
using Microsoft.EntityFrameworkCore;
using MioDolce.Application.Abstractions;
using MioDolce.Application.Common;
using MioDolce.Domain.Common;
using MioDolce.Domain.Suppliers;

namespace MioDolce.Application.Features.Suppliers;

/// <summary>
/// PASSO 7 do guia docs/06: os casos de uso. O controller só chama estes métodos;
/// toda consulta ao banco e toda regra ficam aqui (ou no domínio).
/// </summary>
public sealed class SupplierService(IApplicationDbContext db)
{
    // Projeção reaproveitável: entidade → DTO, traduzida para SQL pelo EF.
    private static readonly Expression<Func<Supplier, SupplierResponse>> ToResponse = s => new SupplierResponse(
        s.Id,
        s.Name,
        s.TaxId,
        s.Email,
        s.Phone,
        s.CreatedAtUtc,
        s.UpdatedAtUtc);

    public async Task<IReadOnlyList<SupplierResponse>> ListAsync(string? search, CancellationToken cancellationToken)
    {
        var suppliers = db.Suppliers.AsNoTracking();

        if (!string.IsNullOrEmpty(search))
        {
            var term = search.Trim();
            var digits = Supplier.NormalizeTaxId(term);
            suppliers = digits.Length > 0
                ? suppliers.Where(s => s.Name.Contains(term) || s.TaxId.Contains(digits))
                : suppliers.Where(s => s.Name.Contains(term));
        }

        return await suppliers
            .OrderBy(s => s.Name)
            .Select(ToResponse)
            .ToListAsync(cancellationToken);
    }
}

```

```

public async Task<Result<SupplierResponse>> GetAsync(Guid id, CancellationToken cancellationToken)
{
    var supplier = await db.Suppliers
        .AsNoTracking()
        .Where(s => s.Id == id)
        .Select(ToResponse)
        .SingleOrDefaultAsync(cancellationToken);

    return supplier is null ? SupplierErrors.NotFound(id) : supplier;
}

public async Task<Result<SupplierResponse>> CreateAsync(SupplierRequest request, CancellationToken cancellationToken)
{
    if (await TaxIdInUseAsync(request.TaxId, exceptId: null, cancellationToken))
    {
        return SupplierErrors.TaxIdAlreadyExists;
    }

    var supplier = Supplier.Create(request.Name, request.TaxId, request.Email, request.Phone);
    db.Suppliers.Add(supplier); // marca como "Added" (INSERT no SaveChanges)
    await db.SaveChangesAsync(cancellationToken);

    return await GetAsync(supplier.Id, cancellationToken);
}

public async Task<Result<SupplierResponse>> UpdateAsync(Guid id, SupplierRequest request, CancellationToken
cancellationToken)
{
    // FindAsync busca pela chave e deixa a entidade RASTREADA: alterar as propriedades gera o UPDATE.
    var supplier = await db.Suppliers.FindAsync([id], cancellationToken);
    if (supplier is null)
    {
        return SupplierErrors.NotFound(id);
    }

    if (await TaxIdInUseAsync(request.TaxId, exceptId: id, cancellationToken))
    {
        return SupplierErrors.TaxIdAlreadyExists;
    }

    supplier.Update(request.Name, request.TaxId, request.Email, request.Phone);

    var saved = await db.SaveChangesAsync(cancellationToken);
    return saved.IsFailure ? saved.Error : await GetAsync(id, cancellationToken);
}

public async Task<Result> DeleteAsync(Guid id, CancellationToken cancellationToken)
{
    var supplier = await db.Suppliers.FindAsync([id], cancellationToken);
    if (supplier is null)
    {
        return SupplierErrors.NotFound(id);
    }

    db.Suppliers.Remove(supplier); // marca como "Deleted" (DELETE no SaveChanges)
    return await db.SaveChangesAsync(cancellationToken);
}

private Task<bool> TaxIdInUseAsync(string taxId, Guid? exceptId, CancellationToken cancellationToken)
{
    var digits = Supplier.NormalizeTaxId(taxId);
    return db.Suppliers.AnyAsync(s => s.TaxId == digits && s.Id != exceptId, cancellationToken);
}
}

```

- **Construtor primário** (`SupplierService(IApplicationDbContext db)`): o parâmetro vira um campo disponível na classe inteira.
- `AsNoTracking()` em leituras: o EF não guarda cópias para detectar mudanças, então é mais rápido.
- **Projeção com** `Select(ToResponse)` : por ser uma `Expression`, o EF traduz para SQL e busca só as colunas do DTO.
- **No "alterar" não existe** `db.Update(...)` : o `FindAsync` devolve a entidade rastreada e o EF percebe sozinho o que mudou.
- `CancellationToken` em tudo: se o cliente desistir da requisição, a consulta ao banco é cancelada.

Um service que altera estado com regra de negócio segue sempre o mesmo roteiro. O registro de movimentação de estoque mostra isso:

```
/// <summary>
/// Fluxo típico de um caso de uso que ALTERA estado:
/// 1. carrega o agregado (rastreado pelo EF);
/// 2. delega a regra de negócio ao domínio;
/// 3. persiste; 4. devolve o contrato de resposta.
/// </summary>
public async Task<Result<StockMovementResponse>> RegisterAsync(
    Guid productId,
    RegisterMovementRequest request,
    CancellationToken cancellationToken)
{
    var product = await db.Products.FindAsync([productId], cancellationToken);
    if (product is null)
    {
        return ProductErrors.NotFound(productId);
    }

    var result = product.RegisterMovement(
        request.Type,
        request.Quantity,
        request.Note,
        timeProvider.GetUtcNow().UtcDateTime);

    if (result.IsFailure)
    {
        return result.Error;
    }

    var movement = result.Value;
    db.StockMovements.Add(movement);

    // Se outra requisição alterou o saldo deste produto no meio do caminho, o
    // ConcurrencyStamp não bate e o UPDATE é rejeitado: nada de saldo negativo por corrida.
    var saved = await db.SaveChangesAsync(cancellationToken);
    if (saved.IsFailure)
    {
        return saved.Error;
    }

    LogMovementRegistered(movement.Type, movement.Delta, product.Sku, movement.BalanceAfter);

    return movement.ToResponseFrom(product);
}
```

9.5 Registro no contêiner de injeção de dependência

Cada camada tem um método `AddXxx()` que registra suas próprias classes. O `Program.cs` só chama esses métodos.

backend/src/MioDolce.Application/DependencyInjection.cs

```
using System.Globalization;
using FluentValidation;
using Microsoft.Extensions.DependencyInjection;
using MioDolce.Application.Features.Categories;
using MioDolce.Application.Features.Dashboard;
using MioDolce.Application.Features.Products;
using MioDolce.Application.Features.Stock;
using MioDolce.Application.Features.Suppliers;

namespace MioDolce.Application;

/// <summary>
/// Cada camada expõe UM método de extensão que registra seus próprios serviços.
/// O Program.cs só chama AddApplication()/AddInfrastructure(), sem conhecer os detalhes.
/// </summary>
public static class DependencyInjection
{
    public static IServiceCollection AddApplication(this IServiceCollection services)
    {
        // Mensagens padrão do FluentValidation em português.
        ValidatorOptions.Global.LanguageManager.Culture = CultureInfo.GetCultureInfo("pt-BR");

        // Registra todos os AbstractValidator<T> deste assembly como IValidator<T>.
        services.AddValidatorsFromAssembly(typeof(DependencyInjection).Assembly, includeInternalTypes: true);
    }
}
```

```

    // Scoped = uma instância por requisição HTTP (mesmo tempo de vida do DbContext).
    services.AddScoped<CategoryService>();
    services.AddScoped<ProductService>();
    services.AddScoped<StockService>();
    services.AddScoped<DashboardService>();
    services.AddScoped<SupplierService>(); // PASSO 8 do guia docs/06: registrar o serviço no DI

    return services;
}
}

```

Tempo de vida	Significa	Use para
AddSingleton	uma instância para a aplicação toda	serviços sem estado por requisição (TimeProvider)
AddScoped	uma instância por requisição HTTP	DbContext e tudo que o usa (services)
AddTransient	uma instância nova a cada injeção	objetos leves e sem estado

Regra de ouro do DI

Um serviço nunca deve depender de outro com tempo de vida **menor**. Um Singleton que recebesse o DbContext (Scoped) guardaria a mesma conexão para sempre.

10 Infrastructure: DbContext e mapeamento

O **Entity Framework Core** (EF Core) é o ORM: traduz objetos C# em linhas de tabela e LINQ em SQL. Três peças fazem isso funcionar, todas **você escreve** .

10.1 O DbContext

Um `DbSet<T>` por tabela. O `OnModelCreating` aplica os mapeamentos de cada entidade, que ficam em arquivos separados.

```

backend/src/MioDolce.Infrastructure/Persistence/AppDbContext.cs

using Microsoft.EntityFrameworkCore;
using MioDolce.Application.Abstractions;
using MioDolce.Domain.Categories;
using MioDolce.Domain.Products;
using MioDolce.Domain.Stock;
using MioDolce.Domain.Suppliers;

namespace MioDolce.Infrastructure.Persistence;

/// <summary>
/// Implementação concreta do banco. O mapeamento de cada entidade fica em
/// Persistence/Configurations (um arquivo por entidade), mantendo o contexto enxuto
/// e o domínio livre de atributos de banco de dados.
/// </summary>
public sealed class AppDbContext : DbContext, IApplicationDbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
        // IMPORTANTE — lição aprendida neste projeto:
        // Desde o EF Core 7, o SaveChanges pode NÃO abrir transação quando o provider julga
        // que o lote de comandos já é atômico. Com o provider MySQL da Oracle isso deixou
        // um INSERT de movimentação gravado mesmo quando o UPDATE do produto falhou por
        // concorrência (saldo e histórico ficavam divergentes).
        // "Always" garante: todos os comandos de um SaveChanges são confirmados juntos ou nenhum.
        Database.AutoTransactionBehavior = AutoTransactionBehavior.Always;
    }

    public DbSet<Category> Categories => Set<Category>();

    public DbSet<Product> Products => Set<Product>();

    public DbSet<StockMovement> StockMovements => Set<StockMovement>();
}

```

```
// PASSO 3 do guia docs/06: um DbSet por tabela que a aplicação consulta.
public DbSet<Supplier> Suppliers => Set<Supplier>();

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Descobre e aplica todas as classes IEntityConfiguration<T> deste assembly.
    modelBuilder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
}
}
```

10.2 Mapeamento: do model para a tabela (Fluent API)

Sem configuração, o EF usa **convenções**: `Id` vira chave primária, `string` vira `NOT NULL`, `string?` vira `NULL`, e o nome da tabela vem do `DbSet`. Mas as convenções não sabem tamanhos, índices, precisão de decimal... Isso vai numa classe de configuração por entidade:

```
backend/src/MioDolce.Infrastructure.Persistence.Configurations/SupplierConfiguration.cs

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using MioDolce.Domain.Suppliers;

namespace MioDolce.Infrastructure.Persistence.Configurations;

/// <summary>
/// PASSO 2 do guia docs/06: como o model vira TABELA.
/// É daqui (e do DbSet no AppDbContext) que o "dotnet ef migrations add" tira as informações.
/// Sem esta classe o EF usaria convenções: varchar(255) para string, nome da tabela = nome do DbSet...
/// </summary>
internal sealed class SupplierConfiguration : IEntityConfiguration<Supplier>
{
    public void Configure(EntityTypeBuilder<Supplier> builder)
    {
        builder.ToTable("suppliers");
        builder.ConfigureAuditTable(); // Id, CreatedAtUtc, UpdatedAtUtc, ConcurrencyStamp

        builder.Property(s => s.Name).HasMaxLength(Supplier.NameMaxLength).IsRequired();

        // IsFixedLength → char(14) em vez de varchar(14): todo CNPJ tem exatamente 14 dígitos.
        builder.Property(s => s.TaxId).HasMaxLength(Supplier.TaxIdLength).IsFixedLength().IsRequired();

        builder.Property(s => s.Email).HasMaxLength(Supplier.EmailMaxLength);
        builder.Property(s => s.Phone).HasMaxLength(Supplier.PhoneMaxLength);

        // Índice único: o banco é a garantia final contra CNPJ duplicado.
        builder.HasIndex(s => s.TaxId).IsUnique();
        builder.HasIndex(s => s.Name);
    }
}
```

Configurações comuns a todas as entidades auditáveis ficam num método de extensão reaproveitável:

```
backend/src/MioDolce.Infrastructure.Persistence.Configurations/AuditTableEntityConfiguration.cs

using Microsoft.EntityFrameworkCore.Metadata.Builders;
using MioDolce.Domain.Common;

namespace MioDolce.Infrastructure.Persistence.Configurations;

internal static class AuditTableEntityConfiguration
{
    /// <summary>Mapeamento comum a toda entidade auditável.</summary>
    public static void ConfigureAuditTable<T>(this EntityTypeBuilder<T> builder)
        where T : AuditTableEntity
    {
        builder.HasKey(e => e.Id);

        // O Guid é gerado no C# (v7), não pelo banco.
        builder.Property(e => e.Id).ValueGeneratedNever();

        builder.Property(e => e.CreatedAtUtc).IsRequired();

        // Entra no WHERE de todo UPDATE/DELETE: "... WHERE Id = @id AND ConcurrencyStamp = @original".
        builder.Property(e => e.ConcurrencyStamp).IsConcurrencyToken();
    }
}
```

```
}  
}
```

O mapeamento de produto mostra mais recursos: enum salvo como texto, precisão de decimal e chave estrangeira:

backend/src/MioDolce.Infrastructure/Persistence/Configurations/ProductConfiguration.cs

```
using Microsoft.EntityFrameworkCore;  
using Microsoft.EntityFrameworkCore.Metadata.Builders;  
using MioDolce.Domain.Products;  
  
namespace MioDolce.Infrastructure.Persistence.Configurations;  
  
internal sealed class ProductConfiguration : IEntityTypeConfiguration<Product>  
{  
    public void Configure(EntityTypeBuilder<Product> builder)  
    {  
        builder.ToTable("products");  
        builder.ConfigureAuditTable();  
  
        builder.Property(p => p.Sku).HasMaxLength(Product.SkuMaxLength).IsRequired();  
        builder.Property(p => p.Name).HasMaxLength(Product.NameMaxLength).IsRequired();  
        builder.Property(p => p.Description).HasMaxLength(Product.DescriptionMaxLength);  
  
        // Enum salvo como texto ("Kilogram") em vez de número: o banco fica legível  
        // e reordenar o enum no C# não corrompe os dados existentes.  
        builder.Property(p => p.Unit).HasConversion<string>().HasMaxLength(20);  
  
        // Sempre defina precisão de decimal. Dinheiro: 2 casas. Quantidade: 3 (ex.: 0,250 kg).  
        builder.Property(p => p.UnitCost).HasPrecision(18, 2);  
        builder.Property(p => p.QuantityOnHand).HasPrecision(18, 3);  
        builder.Property(p => p.MinimumStock).HasPrecision(18, 3);  
  
        builder.HasIndex(p => p.Sku).IsUnique();  
        builder.HasIndex(p => p.Name);  
  
        // Restrict: o banco impede apagar uma categoria que ainda tem produtos.  
        builder.HasOne(p => p.Category)  
            .WithMany()  
            .HasForeignKey(p => p.CategoryId)  
            .OnDelete(DeleteBehavior.Restrict);  
    }  
}
```

Atributos x Fluent API

Daria para escrever `[MaxLength(120)]` e `[Table("suppliers")]` direto no model. Funciona, mas colocaria detalhes de banco dentro do Domain. A Fluent API deixa o Domain limpo e concentra tudo sobre o banco na Infrastructure.

10.3 Interceptor: comportamento automático ao salvar

Um *interceptor* é chamado pelo EF antes de cada `SaveChanges`. Este preenche as datas e renova o token de concorrência de toda entidade auditável, sem nenhum service precisar lembrar disso:

backend/src/MioDolce.Infrastructure/Persistence/Interceptors/AuditableEntityInterceptor.cs · trecho

```
private void Stamp(DbContext? context)  
{  
    if (context is null)  
    {  
        return;  
    }  
  
    var now = timeProvider.GetUtcNow().UtcDateTime;  
  
    foreach (var entry in context.ChangeTracker.Entries<AuditableEntity>())  
    {  
        switch (entry.State)  
        {  
            case EntityState.Added:  
                entry.Property(e => e.CreatedAtUtc).CurrentValue = now;  
                entry.Property(e => e.ConcurrencyStamp).CurrentValue = Guid.NewGuid();  
                break;  
        }  
    }  
}
```

```

        case EntityState.Modified:
            entry.Property(e => e.UpdatedAtUtc).CurrentValue = now;
            // O valor ORIGINAL continua no WHERE; o novo valor vai no SET.
            entry.Property(e => e.ConcurrencyStamp).CurrentValue = Guid.NewGuid();
            break;

        default:
            break;
    }
}
}
}

```

10.4 Registrando o banco no DI

backend/src/MioDolce.Infrastructure/DependencyInjection.cs

```

using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.DependencyInjection.Extensions;
using Microsoft.Extensions.Options;
using MioDolce.Application.Abstractions;
using MioDolce.Infrastructure.Persistence;
using MioDolce.Infrastructure.Persistence.Interceptors;
using MioDolce.Infrastructure.Persistence.Seed;

namespace MioDolce.Infrastructure;

public static class DependencyInjection
{
    public const string ConnectionStringName = "MioDolce";

    public static IServiceCollection AddInfrastructure(this IServiceCollection services)
    {
        // TimeProvider (.NET 8+) substitui DateTime.UtcNow: nos testes dá para "congelar" o relógio.
        services.TryAddSingleton(TimeProvider.System);

        services.AddOptions<DatabaseOptions>()
            .BindConfiguration(DatabaseOptions.SectionName)
            .ValidateOnStart();

        services.AddSingleton<AuditableEntityInterceptor>();

        services.AddDbContext<AppDbContext>((serviceProvider, options) =>
        {
            // Lido de forma "preguiçosa" (quando o DbContext é criado) para que testes
            // e variáveis de ambiente consigam sobrescrever a connection string.
            var configuration = serviceProvider.GetRequiredService<IConfiguration>();
            var connectionString = configuration.GetConnectionString(ConnectionStringName)
                ?? throw new InvalidOperationException(
                    $"Connection string '{ConnectionStringName}' não configurada. Veja backend/README.md.");

            options
                .UseMySQL(connectionString)
                .AddInterceptors(serviceProvider.GetRequiredService<AuditableEntityInterceptor>());

            var database = serviceProvider.GetRequiredService<IOptions<DatabaseOptions>>().Value;
            if (database.SeedSampleData)
            {
                var timeProvider = serviceProvider.GetRequiredService<TimeProvider>();
                options
                    .UseSeeding((context, _) => InventorySeeder.Seed(context, timeProvider))
                    .UseAsyncSeeding((context, _, ct) => InventorySeeder.SeedAsync(context, timeProvider, ct));
            }
        });

        // A Application pede IApplicationDbContext; entregamos a MESMA instância scoped do AppDbContext.
        services.AddScoped<IApplicationDbContext>(sp => sp.GetRequiredService<AppDbContext>());

        services.AddHealthChecks().AddDbContextCheck<AppDbContext>("database");

        return services;
    }
}

```

- `UseMySQL(connectionString)` escolhe o provider. Trocar de banco seria mudar esta linha e o pacote.
- A connection string é lida **quando o DbContext é criado**, para testes e variáveis de ambiente poderem trocá-la.
- `AddScoped<IApplicationDbContext>(sp => sp.GetRequiredService<AppDbContext>())` : quem pede a interface recebe a mesma instância do contexto.
- `UseSeeding` / `UseAsyncSeeding` (EF Core 9+) populam o banco com dados de exemplo quando as migrations são aplicadas.

Um bug real deste projeto: transação

O construtor do `AppDbContext` define `AutoTransactionBehavior.Always`. Sem isso, um teste com 20 saídas simultâneas mostrou que, quando o `UPDATE` do saldo falhava por concorrência, o `INSERT` da movimentação ficava gravado. Com a transação sempre ligada, os comandos de um `SaveChanges` são confirmados juntos ou nenhum é.

11 Migrations

Uma **migration** é uma classe C# que leva o banco de um estado para o próximo. O EF gera a classe comparando seu código (models + configurações + DbSet) com a última "foto" do modelo. Você **não** escreve SQL de criação de tabela.

11.1 O ciclo

1	Você	cria ou altera o model, a configuração e o DbSet
2	Você	roda <code>dotnet ef migrations add NomeDaMudanca</code>
3	EF Core	compara com o snapshot e gera <code>Up()</code> e <code>Down()</code>
4	Você	lê a migration gerada (e, se quiser, o SQL com <code>migrations script</code>)
5	API / você	aplica: a API aplica sozinha ao subir em Development, ou <code>dotnet ef database update</code>
6	EF Core	executa o <code>Up()</code> e registra a migration na tabela <code>__EFMigrationsHistory</code>

11.2 Criando uma migration

```
$ cd backend
$ dotnet ef migrations add AddSuppliers \
  --project src/MioDolce.Infrastructure \
  --startup-project src/MioDolce.Api \
  --output-dir Persistence/Migrations
```

Opção	Significa
<code>--project</code>	onde estão o DbContext e as migrations (Infrastructure)
<code>--startup-project</code>	projeto que o EF executa para montar o DbContext com a configuração real (a Api, que tem o <code>Program.cs</code>)
<code>--output-dir</code>	pasta das migrations (só precisa na primeira; as seguintes vão para a mesma pasta)

11.3 O que é gerado

Arquivo	Função	Pode editar?
<code>20260928194753_AddSuppliers.cs</code>	<code>Up()</code> aplica, <code>Down()</code> desfaz	sim, com cuidado, antes de aplicar
<code>20260928194753_AddSuppliers.Designer.cs</code>	metadados daquela migration	não
<code>AppDbContextModelSnapshot.cs</code>	"foto" do modelo inteiro, base da próxima comparação	nunca

A migration gerada para fornecedores `backend/src/MioDolce.Infrastructure/Persistence/Migrations/20260928194753_AddSuppliers.cs`

```
using System;
using Microsoft.EntityFrameworkCore.Migrations;

#nullable disable
```

```

namespace MioDolce.Infrastructure.Persistence.Migrations
{
    /// <inheritdoc />
    public partial class AddSuppliers : Migration
    {
        /// <inheritdoc />
        protected override void Up(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.CreateTable(
                name: "suppliers",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "char(36)", nullable: false),
                    Name = table.Column<string>(type: "varchar(120)", maxLength: 120, nullable: false),
                    TaxId = table.Column<string>(type: "char(14)", fixedLength: true, maxLength: 14, nullable: false),
                    Email = table.Column<string>(type: "varchar(254)", maxLength: 254, nullable: true),
                    Phone = table.Column<string>(type: "varchar(20)", maxLength: 20, nullable: true),
                    CreatedAtUtc = table.Column<DateTime>(type: "datetime(6)", nullable: false),
                    UpdatedAtUtc = table.Column<DateTime>(type: "datetime(6)", nullable: true),
                    ConcurrencyStamp = table.Column<Guid>(type: "char(36)", nullable: false)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_suppliers", x => x.Id);
                })
                .Annotation("MySQL:Charset", "utf8mb4");

            migrationBuilder.CreateIndex(
                name: "IX_suppliers_Name",
                table: "suppliers",
                column: "Name");

            migrationBuilder.CreateIndex(
                name: "IX_suppliers_TaxId",
                table: "suppliers",
                column: "TaxId",
                unique: true);
        }

        /// <inheritdoc />
        protected override void Down(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.DropTable(
                name: "suppliers");
        }
    }
}

```

Repare como cada configuração do capítulo 10 virou algo concreto: `HasMaxLength(120)` virou `varchar(120)`, `IsFixedLength()` virou `char(14)`, o `HasIndex(...).IsUnique()` virou `CreateIndex(..., unique: true)`.

11.4 Aplicando

Em Development, a API aplica as migrations pendentes ao subir:

```
backend/src/MioDolce.Api/Extensions/DatabaseExtensions.cs
```

```

using Microsoft.EntityFrameworkCore;
using MioDolce.Infrastructure.Persistence;

namespace MioDolce.Api.Extensions;

internal static class DatabaseExtensions
{
    /// <summary>
    /// Aplica as migrations pendentes na inicialização. Prático em desenvolvimento;
    /// em produção prefira rodar as migrations no pipeline de deploy (script SQL ou bundle),
    /// para não depender de várias instâncias da API disputando a mesma migração.
    /// </summary>
    public static async Task ApplyMigrationsAsync(this WebApplication app)
    {
        // O DbContext é Scoped; fora de uma requisição precisamos criar um escopo manualmente.
        await using var scope = app.Services.CreateAsyncScope();
        var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
        await db.Database.MigrateAsync();
    }
}

```

```
}  
}
```

Para aplicar sem subir a API, ou em outro ambiente:

```
$ dotnet ef database update --project src/MioDolce.Infrastructure --startup-project src/MioDolce.Api
```

11.5 Comandos do dia a dia

Todos levam `--project src/MioDolce.Infrastructure --startup-project src/MioDolce.Api`, omitido abaixo para caber:

Quero...	Comando
listar migrations e ver quais estão aplicadas	<code>dotnet ef migrations list</code>
ver o SQL de um intervalo	<code>dotnet ef migrations script InitialCreate AddSuppliers</code>
descartar a última migration não aplicada	<code>dotnet ef migrations remove</code>
voltar o banco para uma migration anterior	<code>dotnet ef database update InitialCreate</code>
voltar o banco ao zero	<code>dotnet ef database update 0</code>
saber se esqueci de gerar uma migration	<code>dotnet ef migrations has-pending-model-changes</code>
script SQL para produção	<code>dotnet ef migrations script --idempotent -o migrate.sql</code>

11.6 Regras que evitam dor de cabeça

1. **Uma migration por mudança**, com nome descritivo: `AddSuppliers`, `AddBarcodeToProducts`.
2. **Nunca edite uma migration já aplicada** no banco de outra pessoa ou de produção. Crie uma nova.
3. Errou e ainda não aplicou? `migrations remove`. Já aplicou só no seu banco? Primeiro `database update <anterior>`, depois `migrations remove`.
4. **Renomear propriedade** pode virar "apagar coluna + criar coluna" e perder dados. Confira se o `Up()` usa `RenameColumn`; se não, ajuste antes de aplicar.
5. O aviso *"An operation was scaffolded that may result in the loss of data"* é para ler com atenção, não para ignorar.
6. Em produção, prefira aplicar pelo pipeline de deploy (script ou bundle) em vez de a API migrar sozinha ao subir.

12 Api: Program.cs, controllers e filtros

12.1 Program.cs: serviços e pipeline

O `Program.cs` tem duas fases. Na primeira, registra serviços no contêiner de DI (a ordem não importa). Na segunda, monta o **pipeline de middlewares**: cada requisição passa por eles de cima para baixo, e aqui **a ordem importa**.

```
backend/src/MioDolce.Api/Program.cs
```

```
using MioDolce.Api;  
using MioDolce.Api.Extensions;  
using MioDolce.Application;  
using MioDolce.Infrastructure;  
using Scalar.AspNetCore;  
  
// — 1. Registro de serviços (injeção de dependência) —————  
// Cada camada sabe registrar o que é seu. A ordem aqui não importa.  
var builder = WebApplication.CreateBuilder(args);  
  
builder.Services  
    .AddApplication()  
    .AddInfrastructure()  
    .AddPresentation(); // inclui AddControllers()  
  
var app = builder.Build();  
  
// — 2. Pipeline de middlewares —————
```

```
// Aqui a ordem IMPORTA: cada requisição atravessa os middlewares de cima para baixo.
app.UseExceptionHandler(); // captura exceções não tratadas → ProblemDetails (500)
app.UseStatusCodePages(); // respostas vazias de erro (ex.: 405) também viram ProblemDetails

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi(); // /openapi/v1.json
    app.MapScalarApiReference(); // /scalar → documentação interativa
    await app.ApplyMigrationsAsync(); // cria/atualiza o schema do banco automaticamente
}
else
{
    app.UseHsts();
    app.UseHttpsRedirection();
}

app.UseCors();

// — 3. Endpoints —
app.MapHealthChecks("/health");
app.MapControllers(); // registra as rotas definidas pelos atributos [Route]/[HttpGet]... dos controllers

await app.RunAsync();
```

12.2 Serviços da camada HTTP

AddControllers e as configurações do MVC

backend/src/MioDolce.Api/DependencyInjection.cs · trecho

```
/// <summary>Serviços da camada HTTP: controllers, JSON, erros, documentação e CORS.</summary>
public static IServiceCollection AddPresentation(this IServiceCollection services)
{
    // AddControllers registra o MVC "só API" (sem Views/Razor). Os controllers são
    // descobertos automaticamente: toda classe pública que herda de ControllerBase.
    services
        .AddControllers(options =>
        {
            // Com Nullable habilitado, o MVC trataria toda string não anulável como [Required]
            // e responderia 400 em inglês ANTES do FluentValidation. Desligamos para que a
            // validação tenha um dono só (FluentValidation, com mensagens em português).
            options.SuppressImplicitRequiredAttributeForNonNullableReferenceTypes = true;

            // Filtro global: vale para todas as actions de todos os controllers.
            options.Filters.Add<FluentValidationActionFilter>();
        })
        // Enums trafegam como texto ("Entry") em vez de número (0): contrato mais legível e estável.
        // ATENÇÃO: controllers usam as opções de JSON do MVC (AddJsonOptions), não as de
        // ConfigureHttpJsonOptions. São configurações separadas.
        .AddJsonOptions(options => options.JsonSerializerOptions.Converters.Add(new JsonStringEnumConverter()))
        // Quando o MODEL BINDING falha (JSON malformatado, "page=abc", enum inexistente), o
        // [ApiController] responde 400 sozinho, antes da action e antes do nosso filtro.
        // InvalidModelStateResponseFactory personaliza essa resposta automática.
        .ConfigureApiBehaviorOptions(options => options.InvalidModelStateResponseFactory = context =>
        {
            var factory = context.HttpContext.RequestServices.GetRequiredService<ProblemDetailsFactory>();
            var problem = factory.CreateValidationProblemDetails(
                context.HttpContext,
                context.ModelState,
                StatusCodes.Status400BadRequest,
                title: "Requisição inválida",
                detail: "Não foi possível ler os dados enviados. Confira o formato do JSON e dos parâmetros.");

            return new BadRequestObjectResult(problem);
        });

    // O gerador de OpenAPI lê as opções de JSON "HTTP": configuramos também para o
    // documento mostrar os enums como texto, igual ao que a API realmente envia.
    services.ConfigureHttpJsonOptions(options =>
        options.SerializerOptions.Converters.Add(new JsonStringEnumConverter()));

    // ProblemDetails (RFC 9457): formato padrão de erro para TODAS as respostas de falha.
    services.AddProblemDetails(options =>
        options.CustomizeProblemDetails = context =>
        {
            context.ProblemDetails.Instance = $"{{context.HttpContext.Request.Method}} {{context.HttpContext.Request.Path}}";
            context.ProblemDetails.Extensions.TryAdd("traceId", context.HttpContext.TraceIdentifier);
        });
}
```

```
services.AddExceptionHandler<GlobalExceptionHandler>();
```

Configuração	Por que existe
<code>AddControllers()</code>	registra o MVC "só API". Descobre sozinho toda classe pública que herda de <code>ControllerBase</code> .
<code>SuppressImplicitRequired...</code>	com <code>Nullable</code> ligado, o MVC trataria toda <code>string</code> como obrigatória e responderia 400 em inglês antes do <code>FluentValidation</code> .
<code>Filters.Add<FluentValidationActionFilter>()</code>	filtro global que roda os validadores antes de toda action.
<code>AddJsonOptions(... JsonStringEnumConverter)</code>	enums viajam como texto (<code>"Kilogram"</code>) em vez de número.
<code>InvalidModelStateResponseFactory</code>	personaliza o 400 automático de JSON malformado.
<code>AddProblemDetails</code> , <code>AddExceptionHandler</code>	tudo erro sai no formato padrão RFC 9457.
<code>AddOpenApi</code>	gera <code>/openapi/v1.json</code> ; o <code>Scalar</code> desenha a interface em <code>/scalar</code> .

12.3 A classe base dos controllers

Todos os controllers herdam dela. Ela traz o `[ApiController]` e o único método que converte erro de negócio em status HTTP:

```
backend/src/MioDolce.Api/Controllers/ApiControllerBase.cs
```

```
using Microsoft.AspNetCore.Mvc;
using MioDolce.Domain.Common;

namespace MioDolce.Api.Controllers;

/// <summary>
/// Classe base de todos os controllers da API.
///
/// - <see cref="ControllerBase"/> (e não <c>Controller</c>): API não renderiza Views,
/// então não precisamos do suporte a Razor que <c>Controller</c> traz.
/// - <see cref="ApiControllerAttribute"/> liga comportamentos de API: exige roteamento por atributo,
/// devolve 400 automático se o model binding falhar (ex.: JSON inválido), infere a origem dos
/// parâmetros ([FromBody], [FromRoute]...) e padroniza erros como ProblemDetails.
/// Colocado na base, vale para todos os controllers que herdam dela.
/// </summary>
[ApiController]
[Produces("application/json")]
public abstract class ApiControllerBase : ControllerBase
{
    /// <summary>
    /// Traduz um erro de negócio (Result) para a resposta HTTP correta.
    /// É o ÚNICO lugar da API que sabe que NotFound vira 404, Conflict vira 409 etc.
    /// </summary>
    protected ObjectResult Problem(Error error)
    {
        var (status, title) = error.Type switch
        {
            ErrorType.Validation => (StatusCodes.Status400BadRequest, "Dados inválidos"),
            ErrorType.NotFound => (StatusCodes.Status404NotFound, "Não encontrado"),
            ErrorType.Conflict => (StatusCodes.Status409Conflict, "Conflito"),
            ErrorType.BusinessRule => (StatusCodes.Status422UnprocessableEntity, "Regra de negócio violada"),
            _ => (StatusCodes.Status500InternalServerError, "Erro interno"),
        };

        // A fábrica aplica os padrões do ASP.NET (type, traceId) e o CustomizeProblemDetails do DI.
        var problem = ProblemDetailsFactory.CreateProblemDetails(HttpContext, status, title, detail: error.Description);
        problem.Extensions["code"] = error.Code;

        return new ObjectResult(problem) { StatusCode = status };
    }
}
```

12.4 Um controller completo

backend/src/MioDolce.Api/Controllers/SuppliersController.cs

```
using Microsoft.AspNetCore.Mvc;
using MioDolce.Application.Features.Suppliers;

namespace MioDolce.Api.Controllers;

/// <summary>
/// PASSO 9 do guia docs/06: o CONTROLLER.
///
/// Checklist de um controller de API:
/// [x] herda de ApiControllerBase (que tem [ApiController] e o helper Problem(Error));
/// [x] [Route] com o prefixo do recurso;
/// [x] uma action por operação, com o verbo HTTP no atributo ([HttpGet], [HttpPost]...);
/// [x] recebe o serviço pelo construtor e NÃO acessa o banco diretamente;
/// [x] devolve status corretos (200, 201 + Location, 204, 400, 404, 409);
/// [x] [ProducesResponseType] documenta cada status para o OpenAPI/Scalar.
/// Não há nada para registrar: AddControllers() + MapControllers() descobrem a classe sozinhos.
/// </summary>
[Route("api/suppliers")]
[Tags("Fornecedores")]
public sealed class SuppliersController(SupplierService service) : ControllerBase
{
    // GET api/suppliers?search=cacau
    // Tipo simples (string) é inferido como query string; [FromQuery] aqui só deixa explícito.
    [HttpGet]
    [EndpointSummary("Lista fornecedores, com busca opcional por nome ou CNPJ")]
    [ProducesResponseType<IReadOnlyList<SupplierResponse>>(StatusCodes.Status200OK)]
    public async Task<ActionResult<IReadOnlyList<SupplierResponse>>> List(
        [FromQuery] string? search,
        CancellationToken cancellationToken) =>
        Ok(await service.ListAsync(search, cancellationToken));

    // GET api/suppliers/{id}
    [HttpGet("{id:guid}")]
    [EndpointSummary("Detalha um fornecedor")]
    [ProducesResponseType<SupplierResponse>(StatusCodes.Status200OK)]
    [ProducesResponseType<ProblemDetails>(StatusCodes.Status404NotFound)]
    public async Task<ActionResult<SupplierResponse>> GetById(Guid id, CancellationToken cancellationToken)
    {
        var result = await service.GetAsync(id, cancellationToken);
        return result.IsSuccess ? result.Value : Problem(result.Error);
    }

    // POST api/suppliers (corpo: SupplierRequest em JSON)
    [HttpPost]
    [EndpointSummary("Cadastra um fornecedor")]
    [ProducesResponseType<SupplierResponse>(StatusCodes.Status201Created)]
    [ProducesResponseType<ValidationProblemDetails>(StatusCodes.Status400BadRequest)]
    [ProducesResponseType<ProblemDetails>(StatusCodes.Status409Conflict)]
    public async Task<ActionResult<SupplierResponse>> Create(SupplierRequest request, CancellationToken cancellationToken)
    {
        var result = await service.CreateAsync(request, cancellationToken);
        return result.IsSuccess
            ? CreatedAtAction(nameof(GetById), new { id = result.Value.Id }, result.Value)
            : Problem(result.Error);
    }

    // PUT api/suppliers/{id} (id da rota + corpo JSON)
    [HttpPut("{id:guid}")]
    [EndpointSummary("Edita um fornecedor")]
    [ProducesResponseType<SupplierResponse>(StatusCodes.Status200OK)]
    [ProducesResponseType<ValidationProblemDetails>(StatusCodes.Status400BadRequest)]
    [ProducesResponseType<ProblemDetails>(StatusCodes.Status404NotFound)]
    [ProducesResponseType<ProblemDetails>(StatusCodes.Status409Conflict)]
    public async Task<ActionResult<SupplierResponse>> Update(Guid id, SupplierRequest request, CancellationToken
cancellationToken)
    {
        var result = await service.UpdateAsync(id, request, cancellationToken);
        return result.IsSuccess ? result.Value : Problem(result.Error);
    }

    // DELETE api/suppliers/{id}
    [HttpDelete("{id:guid}")]
    [EndpointSummary("Exclui um fornecedor")]
}
```

```
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType<ProblemDetails>(StatusCodes.Status404NotFound)]
public async Task<IActionResult> Delete(Guid id, CancellationToken cancellationToken)
{
    var result = await service.DeleteAsync(id, cancellationToken);
    return result.IsSuccess ? NoContent() : Problem(result.Error);
}
```

Anatomia

Peça	Para que serve
<code>[Route("api/suppliers")]</code>	prefixo de todas as rotas do controller
<code>[HttpGet]</code> , <code>[HttpPost]</code> , <code>[HttpPut("{id:guid}")]</code> ...	verbo HTTP + trecho da rota de cada action
<code>{id:guid}</code>	restrição de rota: <code>/api/suppliers/abc</code> vira 404 antes de chegar na action
construtor (<code>SupplierService service</code>)	dependências vindas do DI. Uma instância nova do controller por requisição
<code>ActionResult<T></code>	a action pode devolver o valor (vira 200) ou um resultado de erro
<code>[ProducesResponseType]</code> , <code>[EndpointSummary]</code> , <code>[Tags]</code>	documentação que aparece no Scalar

De onde vem cada parâmetro (model binding)

Parâmetro	Origem com <code>[ApiController]</code>	Exemplo
nome presente na rota	rota	<code>Guid id</code> em <code>[HttpGet("{id:guid}")]</code>
tipo simples (string, int, Guid)	query string	<code>string? search</code> vira <code>?search=</code>
tipo complexo (record, classe)	corpo JSON	<code>SupplierRequest request</code>
record de filtros	query, só com <code>[FromQuery]</code>	<code>[FromQuery] ProductListQuery query</code>
<code>CancellationToken</code>	cancelado se o cliente desistir	sempre o último parâmetro

Respostas

No código	HTTP	Quando
<code>return valor; ou Ok(valor)</code>	200	leitura ou edição com sucesso
<code>CreatedAtAction(nameof(GetById), new { id }, valor)</code>	201 + <code>Location</code>	criação
<code>NoContent()</code>	204	exclusão
<code>Problem(result.Error)</code>	400 / 404 / 409 / 422	erro de negócio

Pegadinha: não use o sufixo Async no nome das actions

O MVC remove "Async" do nome das actions. Com uma action `GetByIdAsync`, o `CreatedAtAction(nameof(GetByIdAsync), ...)` não encontraria a rota e a criação daria erro 500.

12.5 O filtro de validação

O pipeline de uma requisição MVC é: roteamento, model binding, `[ApiController]` (400 se o JSON não pôde ser lido), **action filters**, action, resultado. O filtro procura um validador para cada argumento e, se houver erro, preenche `context.Result`: a action nem executa.

backend/src/MioDolce.Api/Filters/FluentValidationActionFilter.cs · trecho

```
internal sealed class FluentValidationActionFilter(
    IServiceProvider services,
    ProblemDetailsFactory problemDetailsFactory) : IAsyncActionFilter
{
    public async Task OnActionExecutionAsync(ActionExecutingContext context, ActionExecutionDelegate next)
```

```

{
    foreach (var argument in context.ActionArguments.Values)
    {
        if (argument is null)
        {
            continue;
        }

        // Monta o tipo IValidator<TipoDoArgumento> em tempo de execução e pede ao DI.
        var validatorType = typeof(IValidator<>).MakeGenericType(argument.GetType());
        if (services.GetService(validatorType) is not IValidator validator)
        {
            continue; // argumento sem validador (ex.: Guid id, CancellationToken)
        }

        var result = await validator.ValidateAsync(
            new ValidationContext<object>(argument),
            context.HttpContext.RequestAborted);

        foreach (var error in result.Errors)
        {
            // camelCase para bater com os nomes do JSON que o cliente enviou.
            context.ModelState.AddModelError(JsonNamingPolicy.CamelCase.ConvertName(error.PropertyName),
            error.ErrorMessage);
        }

        if (!context.ModelState.IsValid)
        {
            var problem = problemDetailsFactory.CreateValidationProblemDetails(
                context.HttpContext,
                context.ModelState,
                StatusCodes.Status400BadRequest,
                title: "Há campos inválidos");

            // Definir context.Result "curto-circuito" o pipeline: a action não executa.
            context.Result = new BadRequestObjectResult(problem);
            return;
        }

        await next();
    }
}

```

O resultado para o cliente é um **ValidationProblemDetails**, com os erros por campo:

```

POST /api/suppliers { "name": "", "taxId": "123", "email": "x" }

400 Bad Request
{
  "title": "Há campos inválidos",
  "status": 400,
  "errors": {
    "name": ["'Nome' deve ser informado."],
    "taxId": ["'0 CNPJ deve ter 14 dígitos."],
    "email": ["'E-mail' é um endereço de email inválido."]
  },
  "traceId": "00-2f97b92b..."
}

```

12.6 Erros inesperados

O que não é regra de negócio (banco fora do ar, bug) vira exceção, que o `GlobalExceptionHandler` captura: registra no log e devolve 500 **sem** expor stack trace nem SQL ao cliente.

backend/src/MioDolce.Api/Infrastructure/GlobalExceptionHandler.cs · trecho

```

internal sealed partial class GlobalExceptionHandler(
    IProblemDetailsService problemDetailsService,
    ILogger<GlobalExceptionHandler> logger) : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext httpContext, Exception exception, CancellationToken
    cancellationToken)
    {

```

```

var problem = exception switch
{
    // Ex.: corpo acima do limite de tamanho. (JSON malformado em controllers é tratado
    // antes, pelo [ApiController]; veja InvalidModelStateResponseFactory.)
    BadRequestException badRequest => new ProblemDetails
    {
        Status = badRequest.StatusCode,
        Title = "Requisição inválida",
        Detail = badRequest.Message,
    },
    _ => new ProblemDetails
    {
        Status = StatusCodes.Status500InternalServerError,
        Title = "Erro interno",
        Detail = "Ocorreu um erro inesperado. Tente novamente em instantes.",
    },
};

if (problem.Status >= StatusCodes.Status500InternalServerError)
{
    LogUnhandledException(exception, httpContext.Request.Method, httpContext.Request.Path);
}

httpContext.Response.StatusCode = problem.Status!.Value;

return await problemDetailsService.TryWriteAsync(new ProblemDetailsContext
{
    HttpContext = httpContext,
    Exception = exception,
    ProblemDetails = problem,
});
}

[LoggerMessage(Level = LogLevel.Error, Message = "Erro não tratado em {Method} {Path}")]
private partial void LogUnhandledException(Exception exception, string method, PathString path);
}

```

13 Configuração: appsettings, conexão e segredos

O ASP.NET junta a configuração de várias fontes; **a última vence**:

appsettings.json → appsettings.{Ambiente}.json → user-secrets (só Development) → variáveis de ambiente → linha de comando

backend/src/MioDolce.Api/appsettings.json

```

{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore.Database.Command": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "MioDolce": ""
  },
  "Database": {
    "SeedSampleData": false
  },
  "Frontend": {
    "AllowedOrigins": [ "http://localhost:3000" ]
  }
}

```

backend/src/MioDolce.Api/appsettings.Development.json

```

{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",

```

```

    "Microsoft.EntityFrameworkCore.Database.Command": "Information"
  },
},
"ConnectionStrings": {
  "MioDolce": "Server=localhost;Port=3307;Database=miodolce;User=miodolce;Password=miodolce_dev;"
},
"Database": {
  "SeedSampleData": true
}
}
}

```

Porta e ambiente ao rodar com dotnet run

backend/src/MioDolce.Api/Properties/LaunchSettings.json

```

{
  "$schema": "https://json.schemastore.org/launchsettings.json",
  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "scalar",
      "applicationUrl": "http://localhost:5080",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}

```

13.1 Connection string e segredos

A senha de desenvolvimento no `appsettings.Development.json` é aceitável porque o banco é local e descartável. Para qualquer segredo real, use uma destas formas, que **não** vão para o git:

```

# user-secrets: fica no seu perfil do Windows, só vale em Development
$ dotnet user-secrets set "ConnectionStrings:MioDolce" "Server=...;Password=..." --project src/MioDolce.Api
# variável de ambiente: "__" (dois underscores) separa as seções
$ $env:ConnectionStrings__MioDolce = "Server=...;Password=..." # PowerShell

```

13.2 Options pattern: configuração tipada

Em vez de ler `configuration["Database:SeedSampleData"]` espalhado pelo código, cria-se uma classe ligada a uma seção do `appsettings`:

backend/src/MioDolce.Infrastructure/Persistence/DatabaseOptions.cs

```

namespace MioDolce.Infrastructure.Persistence;

/// <summary>
/// Options pattern: uma classe fortemente tipada ligada à seção "Database" do appsettings.
/// Em vez de espalhar configuration["Database:SeedSampleData"] pelo código,
/// injetamos IOptions<DatabaseOptions> onde for preciso.
/// </summary>
public sealed class DatabaseOptions
{
    public const string SectionName = "Database";

    /// <summary>Popula o banco com dados de exemplo quando as migrations são aplicadas.</summary>
    public bool SeedSampleData { get; init; }
}

```

backend/src/MioDolce.Api/Infrastructure/FrontendOptions.cs · trecho

```

internal sealed class FrontendOptions
{
    public const string SectionName = "Frontend";

    [Required]
    [MinLength(1)]

```

```
public string[] AllowedOrigins { get; init; } = [];  
}
```

Registro: `services.AddOptions<FrontendOptions>().BindConfiguration("Frontend").ValidateDataAnnotations().ValidateOnStart()`. Com `ValidateOnStart`, uma configuração inválida impede a API de subir, em vez de falhar no meio de uma requisição. Uso: injete `IOptions<FrontendOptions>` e leia `.Value`.

14 Testes

Tipo	Projeto	Testa	Precisa de
Unidade	MioDolce.Domain.Tests	regras dos models, isoladas	nada (roda em milissegundos)
Integração	MioDolce.Api.IntegrationTests	HTTP, controller, service, EF e MySQL juntos	Docker rodando

14.1 Teste de unidade

Padrão AAA: **Arrange** (prepara), **Act** (executa), **Assert** (confere). Nome: `Metodo_Cenario_ResultadoEsperado`.

backend/tests/MioDolce.Domain.Tests/Products/ProductTests.cs · trecho

```
[Fact]
public void RegisterMovement_ExitBeyondBalance_FailsAndKeepsBalance()
{
    var product = NewProductWithBalance(3);

    var result = product.RegisterMovement(MovementType.Exit, 5, null, Now);

    Assert.True(result.IsFailure);
    Assert.Equal("Product.InsufficientStock", result.Error.Code);
    Assert.Equal(3, product.QuantityOnHand);
}
```

14.2 Teste de integração

A `ApiFactory` sobe a API inteira em memória (`WebApplicationFactory`) apontando para um MySQL descartável criado pelo `Testcontainers`:

backend/tests/MioDolce.Api.IntegrationTests/Infrastructure/ApiFactory.cs · trecho

```
public sealed class ApiFactory : WebApplicationFactory<Program>, IAsyncLifetime
{
    public static readonly JsonSerializerOptions Json = new(JsonSerializerDefaults.Web)
    {
        Converters = { new JsonStringEnumConverter() },
    };

    private readonly MySqlContainer _mysql = new MySqlBuilder("mysql:8.4").Build();

    public async ValueTask InitializeAsync() => await _mysql.StartAsync();

    public override async ValueTask DisposeAsync()
    {
        await base.DisposeAsync();
        await _mysql.DisposeAsync();
    }

    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        // Ambiente Development: a API aplica as migrations sozinha ao subir.
        builder.UseEnvironment("Development");
        builder.UseSetting("ConnectionStrings:MioDolce", _mysql.GetConnectionString());
        builder.UseSetting("Database:SeedSampleData", "false");
    }
}
```

```
[Fact]
public async Task Create_DuplicateTaxId_Returns409()
{
    var taxId = UniqueTaxId();
    var first = await _client.PostAsJsonAsync("/api/suppliers", new SupplierRequest("Primeiro", taxId, null, null),
    ApiFactory.Json, Ct);
    Assert.Equal(HttpStatusCode.Created, first.StatusCode);

    var second = await _client.PostAsJsonAsync("/api/suppliers", new SupplierRequest("Segundo", taxId, null, null),
    ApiFactory.Json, Ct);

    Assert.Equal(HttpStatusCode.Conflict, second.StatusCode);
}
```

```
$ dotnet test # todos
$ dotnet test --project tests/MioDolce.Domain.Tests # só os rápidos
$ dotnet test --project tests/MioDolce.Api.IntegrationTests --filter-class "*SuppliersControllerTests"
```

15 Receita: um recurso novo do zero

Resumo de tudo que você programa para criar um recurso (use Fornecedores como gabarito). Cada arquivo do projeto tem um comentário `PASSO N` com a etapa correspondente.

#	O que	Onde	Quem
1	Model + catálogo de erros	Domain/<Recurso>/	você escreve
2	Configuração da tabela	Infrastructure/Persistence/Configurations/	você escreve
3	DbSet no contexto e na interface	AppDbContext.cs , IApplicationDbContext.cs	você escreve
4	Migration	dotnet ef migrations add Add<Recurso>	gerado e revisado
5	Request e response	Application/Features/<Recurso>/...Contracts.cs	você escreve
6	Validador	...RequestValidator.cs	você escreve
7	Service	...Service.cs	você escreve
8	Registrar o service	Application/DependencyInjection.cs	você escreve
9	Controller	Api/Controllers/<Recurso>sController.cs	você escreve
10	Testes de integração	tests/MioDolce.Api.IntegrationTests/	você escreve
11	Requisições de teste manual	MioDolce.Api.http	você escreve

Não é preciso registrar o validador (o DI o encontra), nem o controller (`AddControllers` o encontra), nem a configuração da tabela (`ApplyConfigurationsFromAssembly` a encontra).

Para praticar

Crie "Locais de armazenamento" (`StorageLocation` : nome único, temperatura mínima e máxima opcionais) seguindo a tabela, sem olhar o código de fornecedores até travar. Depois faça o produto apontar para um local com uma chave estrangeira opcional: é uma segunda migration, e dá para comparar como uma FK aparece no `Up()` .

Frontend em Next.js

Next.js 16 com App Router, React 19 e Tailwind CSS v4. O frontend não tem regra de negócio: ele mostra dados da API e envia formulários para ela.

16 Criando o app Next.js

```
$ npx create-next-app@latest frontend --ts --tailwind --eslint --app --src-dir --import-alias "@/*" --use-npm
$ cd frontend
$ npm install @phosphor-icons/react server-only zod
$ npm run dev # http://localhost:3000
```

Opção / pacote	Para quê
<code>--ts --tailwind --eslint</code>	TypeScript, Tailwind CSS v4 e ESLint já configurados
<code>--app --src-dir</code>	App Router (pasta <code>src/app</code>), o modelo atual do Next
<code>--import-alias "@/*"</code>	importar com <code>@/lib/format</code> em vez de <code>../../../lib/format</code>
<code>@phosphor-icons/react</code>	ícones
<code>server-only</code>	faz o build falhar se um arquivo de servidor for importado no navegador
<code>zod</code>	valida as variáveis de ambiente

Depois crie `.env.local` **você escreve** com a URL da API. Ele não vai para o git; o `.env.example` vai, como modelo:

```
API_URL=http://localhost:5080
```

16.1 Estrutura de pastas

<code>src/</code>	
<code>app/</code>	ROTAS: cada pasta é uma URL; <code>page.tsx</code> é a página
<code>features/</code>	código por ASSUNTO (produtos, categorias, movimentações, painel)
<code>products/</code>	
<code>types.ts</code>	tipos que espelham os DTOs do C#
<code>labels.ts</code>	traduções (a API fala inglês, a tela fala português)
<code>api.ts</code>	uma função por endpoint da API (servidor)
<code>actions.ts</code>	Server Actions: gravações (servidor)
<code>components/</code>	formulário, tabela, badges do assunto
<code>components/</code>	UI genérica: <code>ui/</code> (botão, campo, paginação) e <code>layout/</code> (cabeçalho)
<code>lib/</code>	utilitários: cliente HTTP, env, formatação, formulários

Regra: `features/` pode usar `components/` e `lib/`; o contrário nunca. Assim a UI genérica não conhece regras de assunto nenhum.

17 Rotas, layout e estilo

17.1 Pasta = rota

Arquivo	URL	Página
<code>app/page.tsx</code>	<code>/</code>	painel
<code>app/produtos/page.tsx</code>	<code>/produtos</code>	lista com filtros
<code>app/produtos/novo/page.tsx</code>	<code>/produtos/novo</code>	cadastro
<code>app/produtos/[id]/page.tsx</code>	<code>/produtos/0199...</code>	detalhe (<code>[id]</code> = segmento dinâmico)

Arquivo	URL	Página
app/produtos/[id]/editar/page.tsx	/produtos/0199.../editar	edição
app/categorias/page.tsx	/categorias	lista + formulário
app/movimentacoes/page.tsx	/movimentacoes	histórico

Arquivos especiais em qualquer pasta de rota: `layout.tsx` (moldura), `loading.tsx` (aparece enquanto a página busca dados), `error.tsx` (erro inesperado, com botão "tentar de novo") e `not-found.tsx` (404).

17.2 Layout raiz

frontend/src/app/layout.tsx

```
import type { Metadata } from "next";
import { Instrument_Sans, JetBrains_Mono } from "next/font/google";
import { AppHeader } from "@/components/layout/app-header";
import "@/globals.css";

// next/font baixa as fontes no build e as serve localmente (sem requisição ao Google no navegador).
const sans = Instrument_Sans({ subsets: ["latin"], variable: "--font-instrument-sans" });
const mono = JetBrains_Mono({ subsets: ["latin"], variable: "--font-jetbrains-mono" });

export const metadata: Metadata = {
  title: { default: "MioDolce · Estoque", template: "%s · MioDolce" },
  description: "Controle de estoque do almoxarifado da confeitaria.",
};

export default function RootLayout({ children }: LayoutProps<"/>) {
  return (
    <html lang="pt-BR" className={` ${sans.variable} ${mono.variable}`}>
      <body className="min-h-dvh font-sans">
        <a
          href="#conteudo"
          className="sr-only focus:not-sr-only focus:fixed focus:left-4 focus:top-3 focus:z-20 focus:rounded-md focus:bg-surface
focus:px-3 focus:py-2"
        >
          Pular para o conteúdo
        </a>
        <AppHeader />
        <main id="conteudo" className="mx-auto w-full max-w-350 px-4 pb-20 pt-8 md:px-8">
          {children}
        </main>
      </body>
    </html>
  );
}
```

17.3 Server Components e Client Components

Todo componente é **Server Component** por padrão: roda no servidor, pode ser `async` e buscar dados. Só vira **Client Component** (roda também no navegador) com `"use client"` no topo do arquivo. Use apenas quando precisar de interação:

Precisa de...	Tipo	Exemplo no projeto
buscar dados, montar HTML	Server	todas as <code>page.tsx</code> , tabelas, listas
<code>useState</code> , eventos (<code>onClick</code> , <code>hover</code>)	Client	<code>flow-chart.tsx</code> , <code>movement-form.tsx</code>
<code>useActionState</code> , <code>useFormStatus</code>	Client	formulários, <code>submit-button.tsx</code>
rota atual (<code>usePathname</code>)	Client	<code>nav-link.tsx</code>

17.4 Tailwind v4 e tokens de design

As cores são variáveis CSS com nomes de **função** (`--ink`, `--surface`, `--accent`), redefinidas no modo escuro. O bloco `@theme inline` as transforma em classes do Tailwind (`bg-surface`, `text-ink`). Nenhum componente precisa de `dark:`.

```
:root {
  --bg: #f3f4f2;
  --surface: #fbfbfa;
  --sunken: #eceeec;
  --ink: #16191a;
  --ink-2: #41474a;
  --muted: #656b6e;
  --line: #dcdfdb;
  --line-strong: #c4c8c3;

  --accent: #4f6d14;
  --accent-ink: #f6f8ef;
  --accent-soft: #e5ecd3;

  --warn: #8f5304;
  --warn-soft: #faedd6;
  --danger: #b3261e;
  --danger-soft: #fbe5e2;

  /* Séries do gráfico de fluxo (validadas para daltonismo e contraste). */
  --flow-in: #5f7f1c;
  --flow-out: #2f66b3;
}
```

```
@theme inline {
  --color-bg: var(--bg);
  --color-surface: var(--surface);
  --color-sunken: var(--sunken);
  --color-ink: var(--ink);
  --color-ink-2: var(--ink-2);
  --color-muted: var(--muted);
  --color-line: var(--line);
  --color-line-strong: var(--line-strong);
  --color-accent: var(--accent);
  --color-accent-ink: var(--accent-ink);
  --color-accent-soft: var(--accent-soft);
  --color-warn: var(--warn);
  --color-warn-soft: var(--warn-soft);
  --color-danger: var(--danger);
  --color-danger-soft: var(--danger-soft);
  --color-flow-in: var(--flow-in);
  --color-flow-out: var(--flow-out);

  --font-sans: var(--font-instrument-sans);
  --font-mono: var(--font-jetbrains-mono);
}
```

18 Acesso à API

Três camadas, todas **você escreve**, todas rodando só no servidor.

18.1 Variáveis de ambiente validadas

frontend/src/lib/env.ts

```
import "server-only";
import { z } from "zod";

/**
 * Variáveis de ambiente validadas na inicialização.
 * "server-only" garante erro de build se este arquivo for importado em um Client Component:
 * a URL da API nunca vai para o navegador.
 */
const schema = z.object({
  API_URL: z.url().default("http://localhost:5080"),
});

export const env = schema.parse({
```

```
API_URL: process.env.API_URL,
});
```

18.2 O cliente HTTP: o único `fetch` do projeto

Monta a URL, envia JSON, e transforma qualquer resposta de erro em `ApiError` carregando o `ProblemDetails` que o ASP.NET devolveu:

frontend/src/lib/api/client.ts

```
import "server-only";
import { env } from "@lib/env";
import type { ProblemDetails } from "./types";

/**
 * Erro vindo da API. Carrega o ProblemDetails (RFC 9457) que o ASP.NET Core devolve,
 * incluindo `errors` (validação por campo) e `code` (código estável do erro de negócio).
 */
export class ApiError extends Error {
  readonly status: number;
  readonly problem: ProblemDetails;
} {
  super(problem.detail ?? problem.title ?? `Erro ${status} na API`);
  this.name = "ApiError";
}

type RequestOptions = {
  method?: "GET" | "POST" | "PUT" | "DELETE";
  body?: unknown;
  query?: Record<string, string | number | boolean | undefined | null>;
};

/**
 * Único ponto de acesso HTTP à API .NET. Roda só no servidor (Server Components e
 * Server Actions), então o navegador nunca conversa direto com o backend.
 */
export async function apiFetch<T>(path: string, options: RequestOptions = {}): Promise<T> {
  const url = new URL(path, env.API_URL);
  for (const [key, value] of Object.entries(options.query ?? {})) {
    if (value !== undefined && value !== null && value !== "") {
      url.searchParams.set(key, String(value));
    }
  }

  const response = await fetch(url, {
    method: options.method ?? "GET",
    headers: options.body === undefined ? undefined : { "Content-Type": "application/json" },
    body: options.body === undefined ? undefined : JSON.stringify(options.body),
    // Dados de estoque mudam o tempo todo: sempre buscar o valor atual.
    cache: "no-store",
  });

  if (!response.ok) {
    const problem = (await response.json().catch(() => ({}))) as ProblemDetails;
    throw new ApiError(response.status, { status: response.status, ...problem });
  }

  if (response.status === 204) {
    return undefined as T;
  }

  return (await response.json()) as T;
}
```

18.3 Tipos e funções por assunto

Cada response do C# ganha um tipo TypeScript equivalente, e cada endpoint uma função. As páginas nunca montam URL da API:

frontend/src/features/categories/types.ts

```
export type Category = {
  id: string;
```

```

name: string;
description: string | null;
productCount: number;
createdAtUtc: string;
};

export type CategoryRequest = {
  name: string;
  description: string | null;
};

```

frontend/src/features/categories/api.ts

```

import "server-only";
import { apiFetch } from "@lib/api/client";
import type { Category, CategoryRequest } from "../types";

// Uma função por endpoint: as páginas nunca montam URLs da API diretamente.
export const categoriesApi = {
  list: () => apiFetch<Category[]>("/api/categories"),
  get: (id: string) => apiFetch<Category>(`/api/categories/${id}`),
  create: (body: CategoryRequest) => apiFetch<Category>("/api/categories", { method: "POST", body }),
  update: (id: string, body: CategoryRequest) => apiFetch<Category>(`/api/categories/${id}`, { method: "PUT", body }),
  remove: (id: string) => apiFetch<void>(`/api/categories/${id}`, { method: "DELETE" }),
};

```

19 Lendo dados: Server Components

Uma página é uma função `async` que chama a API e devolve JSX. Sem `useEffect` e sem estado de carregamento manual: enquanto ela espera, o `loading.tsx` aparece.

Lista de produtos: filtros vindos da URL, duas chamadas em paralelo

frontend/src/app/produtos/page.tsx

```

import type { Metadata } from "next";
import { PackageIcon } from "@phosphor-icons/react/ssr";
import { EmptyState } from "@components/ui/empty-state";
import { PageHeader } from "@components/ui/page-header";
import { Pagination } from "@components/ui/pagination";
import { SegmentedLinks } from "@components/ui/segmented-links";
import { categoriesApi } from "@features/categories/api";
import { productsApi } from "@features/products/api";
import { ProductFilters } from "@features/products/components/product-filters";
import { ProductTable } from "@features/products/components/product-table";
import { statusLabels } from "@features/products/labels";
import { productSorts, stockStatuses } from "@features/products/types";
import { hrefWith, readEnum, readPage, readString } from "@lib/search-params";

export const metadata: Metadata = { title: "Produtos" };

// PageProps<"/produtos"> é gerado pelo Next a partir da estrutura de pastas.
export default async function ProductsPage({ searchParams }: PageProps<"/produtos">) {
  const params = await searchParams;
  const filters = {
    search: readString(params, "search"),
    categoryId: readString(params, "categoryId"),
    status: readEnum(params, "status", stockStatuses),
    sort: readEnum(params, "sort", productSorts),
    includeInactive: readString(params, "includeInactive") === "true",
  };

  // As duas chamadas são independentes: Promise.all dispara em paralelo (sem "cascata").
  const [result, categories] = await Promise.all([
    productsApi.list({ ...filters, page: readPage(params), pageSize: 20 }),
    categoriesApi.list(),
  ]);

  const statusOptions = [
    { label: "Todos", href: hrefWith("/produtos", params, { status: undefined, page: undefined }), active: !filters.status },
    ...stockStatuses.map((status) => ({
      label: statusLabels[status],
      href: hrefWith("/produtos", params, { status, page: undefined }),
      active: filters.status === status,
    })),
  ],

```

```

];

return (
  <div className="grid gap-6">
    <PageHeader
      title="Produtos"
      description={` ${result.totalCount} ${result.totalCount === 1 ? "produto encontrado" : "produtos encontrados"}.` }
    />

    <div className="grid gap-4">
      <SegmentedLinks label="Filtrar por situação" options={statusOptions} />
      <ProductFilters categories={categories} current={filters} />
    </div>

    {result.items.length === 0 ? (
      <EmptyState icon={<PackageIcon size={28} />} title="Nenhum produto com esses filtros">
        Limpe a busca ou escolha outra situação. Produtos arquivados só aparecem com a opção marcada.
      </EmptyState>
    ) : (
      <div>
        <ProductTable products={result.items} />
        <Pagination pathname="/produtos" params={params} data={result} />
      </div>
    )}
  </div>
);
}

```

- `searchParams` é uma **Promise** no Next 16: sempre `await`.
- `PageProps<"/produtos">` é um tipo gerado a partir das pastas (`npx next typegen`, ou automaticamente no `npm run dev`).
- **Estado na URL**: filtros e página são parâmetros da URL (`?status=Low&page=2`), então o link é compartilhável e o botão voltar funciona.
- `Promise.all`: chamadas independentes em paralelo, sem esperar uma pela outra.

Para uma página de detalhe, `cache()` evita chamar a API duas vezes (página + título da aba) e um 404 da API vira a página "não encontrado":

frontend/src/features/products/load-product.ts

```

import "server-only";
import { cache } from "react";
import { notFound } from "next/navigation";
import { ApiError } from "@lib/api/client";
import { productsApi } from "./api";

/**
 * cache() do React deduplica chamadas na MESMA renderização: generateMetadata e a página
 * pedem o mesmo produto, mas a API é chamada uma vez só. 404 da API vira a página not-found.
 */
export const loadProduct = cache(async (id: string) => {
  try {
    return await productsApi.get(id);
  } catch (error) {
    if (error instanceof ApiError && error.status === 404) notFound();
    throw error;
  }
});

```

20 Gravando dados: Server Actions e formulários

Uma **Server Action** é uma função com `"use server"` que roda no servidor do Next e pode ser ligada direto a um `<form>`. O navegador envia o formulário para o Next (não para a API); a action chama a API.

20.1 O estado do formulário

frontend/src/lib/form-state.ts

```

/**
 * Estado devolvido por toda Server Action de formulário (consumido com useActionState).

```

```

* Fica fora do "server-only" porque os Client Components também usam o tipo e o estado inicial.
* `values` devolve o que o usuário digitou, para o formulário não "esvaziar" após um erro.
*/
export type FormState = {
  status: "idle" | "error";
  message?: string;
  fieldErrors?: Record<string, string[]>;
  values?: Record<string, string>;
};

export const idleState: FormState = { status: "idle" };

```

Convertendo o erro da API em erros por campo

frontend/src/lib/forms.ts · trecho

```

/**
* Converte a falha da API em estado de formulário:
* 400 com `errors` vira erro por campo; o resto vira mensagem geral (detail do ProblemDetails).
*/
export function toFormError(error: unknown, formData: FormData): FormState {
  if (error instanceof ApiError) {
    return {
      status: "error",
      message: error.problem.errors ? "Revise os campos destacados." : error.message,
      fieldErrors: error.problem.errors,
      values: formValues(formData),
    };
  }

  // Rede fora do ar, API desligada etc.: deixa o error.tsx da rota tratar.
  throw error;
}

```

20.2 A Server Action

frontend/src/features/categories/actions.ts

```

"use server";

import { redirect } from "next/navigation";
import { revalidatePath } from "next/cache";
import { ApiError } from "@lib/api/client";
import type { FormState } from "@lib/form-state";
import { toFormError, toOptionalText, toText } from "@lib/forms";
import { categoriesApi } from "../api";

/*
Server Actions: rodam no servidor, recebem o FormData do <form> e chamam a API.
A validação de verdade é da API (FluentValidation); aqui só convertemos os tipos
e repassamos os erros por campo que ela devolve.
*/

export async function saveCategory(_prev: FormState, formData: FormData): Promise<FormState> {
  const id = toText(formData.get("id"));
  const body = {
    name: toText(formData.get("name")),
    description: toOptionalText(formData.get("description")),
  };

  try {
    if (id) await categoriesApi.update(id, body);
    else await categoriesApi.create(body);
  } catch (error) {
    return toFormError(error, formData);
  }

  revalidatePath("/categorias");
  // redirect lança uma exceção de controle; por isso fica FORA do try/catch.
  redirect("/categorias");
}

export async function deleteCategory(_prev: FormState, formData: FormData): Promise<FormState> {
  try {
    await categoriesApi.remove(toText(formData.get("id")));
  } catch (error) {
    if (error instanceof ApiError) return { status: "error", message: error.message };
  }
}

```

```

    throw error;
  }

  revalidatePath("/categorias");
  redirect("/categorias");
}

```

`redirect()` fora do `try/catch`

`redirect` funciona lançando uma exceção de controle. Dentro do `try`, o `catch` a engoliria e o redirecionamento não aconteceria.

20.3 O formulário

frontend/src/features/categories/components/category-form.tsx

```

"use client";

import { useActionState } from "react";
import { ButtonLink } from "@/components/ui/button";
import { describedBy, Field, Input, Textarea } from "@/components/ui/field";
import { FormAlert } from "@/components/ui/form-alert";
import { SubmitButton } from "@/components/ui/submit-button";
import { idleState } from "@/lib/form-state";
import { saveCategory } from "../actions";
import type { Category } from "../types";

export function CategoryForm({ category }: { category?: Category }) {
  const [state, formAction] = useActionState(saveCategory, idleState);
  const errors = state.fieldErrors ?? {};

  return (
    <form action={formAction} className="grid gap-4" noValidate>
      {category ? <input type="hidden" name="id" value={category.id} /> : null}
      <FormAlert message={state.message} />

      <Field label="Nome" htmlFor="name" errors={errors.name}>
        <Input {...describedBy("name", errors.name)} defaultValue={state.values?.name ?? category?.name ?? ""} required />
      </Field>

      <Field label="Descrição" htmlFor="description" hint="Opcional." errors={errors.description}>
        <Textarea
          {...describedBy("description", errors.description, true)}
          defaultValue={state.values?.description ?? category?.description ?? ""}
          rows={3}
        />
      </Field>

      <div className="flex gap-2">
        <SubmitButton>{category ? "Salvar" : "Criar categoria"}</SubmitButton>
        {category ? (
          <ButtonLink href="/categorias" variant="ghost">
            Cancelar
          </ButtonLink>
        ) : null}
      </div>
    </form>
  );
}

```

- `useActionState(action, estadoInicial)` devolve o último estado retornado pela action e a função para o `action` do form.
- Os erros por campo que aparecem abaixo de cada input vêm **do FluentValidation da API**: a regra existe num lugar só.
- `defaultValue={state.values?.name ?? ...}` reexibe o que foi digitado quando há erro.
- O `SubmitButton` usa `useFormStatus` para desabilitar o botão enquanto a action roda.

Pegadinha do React 19 (bug real deste projeto)

Depois que uma Server Action termina, o React **reseta o formulário**. Com radio *controlado* (`checked=`), o DOM voltava para "Entrada" enquanto o estado ainda dizia "Saída": o botão mostrava "Registrar saída" e o formulário enviava uma entrada. A correção é usar `defaultChecked` acompanhando o estado:

frontend/src/features/movements/components/movement-form.tsx · trecho

```
<input
  type="radio"
  name="type"
  value={option}
  defaultChecked={type === option}
  onChange={() => setType(option)}
  className="sr-only"
/>
```

20.4 Receita: uma tela nova

#	Arquivo	Conteúdo
1	features/<assunto>/types.ts	tipos iguais ao response e ao request do C#
2	features/<assunto>/api.ts	uma função por endpoint, usando <code>apiFetch</code>
3	features/<assunto>/actions.ts	Server Actions de criar/editar/excluir
4	features/<assunto>/components/*-form.tsx	formulário com <code>useActionState</code>
5	app/<rota>/page.tsx	página que busca os dados e monta a tela
6	components/layout/app-header.tsx	link no menu

Next.js conversando com o ASP.NET

Os dois são programas independentes que conversam por HTTP e JSON. Esta parte mostra quem chama quem, o formato combinado entre eles e como cada erro da API aparece na tela.

21 O caminho de uma leitura e de uma gravação



21.1 Leitura: abrir /produtos?status=Low

1	Navegador	pede <code>GET localhost:3000/produtos?status=Low</code> ao Next
2	Next	mostra o <code>loading.tsx</code> imediatamente (streaming) e executa <code>ProductsPage</code> no servidor
3	Next	<code>productsApi.list(...)</code> chama <code>apiFetch</code> , que faz <code>GET localhost:5080/api/products?status=Low&pageSize=20</code>
4	API	o <code>ProductsController.List</code> recebe o <code>[FromQuery] ProductListQuery</code> , o filtro valida, o <code>ProductService</code> consulta o MySQL
5	API	responde <code>200</code> com <code>PagedResponse<ProductResponse></code> em JSON
6	Next	renderiza a tabela em HTML e envia ao navegador, substituindo o esqueleto de carregamento

O navegador recebe HTML pronto. Ele nunca vê a URL da API nem o JSON original.

21.2 Gravação: criar uma categoria

1	Navegador	o usuário envia o <code><form action={formAction}></code> ; o React faz um <code>POST</code> para o Next com o <code>FormData</code>
2	Next	executa a Server Action <code>saveCategory</code> , que monta <code>{ name, description }</code>
3	Next	<code>categoriesApi.create</code> faz <code>POST localhost:5080/api/categories</code> com JSON
4	API	binding do JSON em <code>CategoryRequest</code> ; filtro de validação; <code>CategoryService.CreateAsync</code> ; <code>INSERT</code>
5a	API: sucesso	<code>201 Created</code> ; a action chama <code>revalidatePath("/categorias")</code> e <code>redirect("/categorias")</code> : a lista aparece atualizada
5b	API: erro	<code>400</code> ou <code>409</code> com <code>ProblemDetails</code> ; <code>apiFetch</code> lança <code>ApiError</code> ; a action devolve <code>FormState</code> e o formulário mostra as mensagens

22 Contratos, erros e CORS

22.1 O contrato JSON

Os dois lados combinam um formato. O C# define, o TypeScript espelha:

C# (record da API)	JSON trafegado	TypeScript (<code>types.ts</code>)
<code>Guid Id</code>	<code>"id": "0199a0e9-..."</code>	<code>id: string</code>
<code>string Name</code>	<code>"name": "Chocolates"</code> (camelCase)	<code>name: string</code>
<code>string? Description</code>	<code>"description": null</code>	<code>description: string null</code>

C# (record da API)	JSON trafegado	TypeScript (types.ts)
<code>decimal UnitCost</code>	<code>"unitCost": 89.9</code>	<code>unitCost: number</code>
<code>UnitOfMeasure Unit</code> (enum)	<code>"unit": "Kilogram"</code> (texto, por causa do <code>JsonStringEnumConverter</code>)	<code>unit: "Unit" "Kilogram" ...</code>
<code>DateTime CreatedAtUtc</code>	<code>"createdAtUtc": "2026-09-28T18:42:26Z"</code>	<code>createdAtUtc: string</code> (formatado com Intl)
<code>DateOnly Date</code>	<code>"date": "2026-09-28"</code>	<code>date: string</code>

Quando mudar um DTO no C#, mude o `types.ts` correspondente.

Um próximo passo de estudo é gerar os tipos automaticamente a partir do OpenAPI da API: `npx openapi-typescript http://localhost:5080/openapi/v1.json -o src/lib/api/schema.d.ts` .

22.2 Cada resposta da API e o que a tela faz

API responde	Exemplo	Next faz	Onde
200 / 201 / 204	sucesso	mostra os dados, ou <code>revalidatePath</code> + <code>redirect</code>	página / action
400 com <code>errors</code>	nome vazio	mensagem abaixo de cada campo	<code>toFormError</code> + <code>Field</code>
404	produto inexistente	<code>notFound()</code> mostra a página 404	<code>load-product.ts</code>
409	SKU ou CNPJ repetido, concorrência	alerta no topo do formulário com o <code>detail</code>	<code>FormAlert</code>
422	estoque insuficiente	alerta no formulário	<code>FormAlert</code>
500 ou API fora do ar	exceção, rede	tela de erro com "tentar de novo"	<code>app/error.tsx</code>

O campo `code` do `ProblemDetails` (ex.: `"Product.InsufficientStock"`) é estável: o front pode usá-lo para tratar um erro específico sem depender do texto da mensagem.

22.3 E o CORS?

CORS é uma proteção do **navegador**: ele bloqueia uma página de `localhost:3000` que tenta chamar `localhost:5080` , a menos que a API autorize. Neste projeto quem chama a API é o **servidor** do Next, não o navegador, então o CORS nem entra em jogo. A API ainda tem uma política configurada (`Frontend:AllowedOrigins` no `appsettings`) para o dia em que você quiser chamar a API de um Client Component.

22.4 Cache

O `apiFetch` usa `cache: "no-store"` : dado de estoque muda o tempo todo, então cada visita busca o valor atual. Depois de gravar, `revalidatePath` avisa o Next para renderizar de novo as páginas afetadas.

22.5 Portas e endereços

Serviço	Endereço	Configurado em
Frontend	<code>http://localhost:3000</code>	padrão do <code>next dev</code>
API	<code>http://localhost:5080</code>	<code>Properties/launchSettings.json</code>
Documentação da API	<code>http://localhost:5080/scalar</code>	<code>MapScalarApiReference()</code>
MySQL	<code>localhost:3307</code>	<code>docker-compose.yml</code> e a connection string
Front → API	<code>API_URL</code>	<code>frontend/.env.local</code>

Rodando e evoluindo

Os comandos do dia a dia, os problemas mais comuns e o vocabulário usado neste guia.

23 Rodar, testar e resolver problemas

23.1 Subindo tudo (três terminais)

```
# 1. banco
$ docker compose up -d
# 2. API: aplica migrations e popula o banco na primeira vez
$ cd backend
$ dotnet run --project src/MioDolce.Api
# 3. frontend
$ cd frontend
$ npm install
$ npm run dev
```

23.2 Verificações antes de dar algo como pronto

```
$ cd backend && dotnet build && dotnet test
$ cd frontend && npx tsc --noEmit && npm run lint && npm run build
```

23.3 Problemas comuns

Sintoma	Causa provável	Solução
No .NET SDKs were found	só o runtime está instalado	<code>winget install Microsoft.DotNet.SDK.10</code>
ports are not available: 3306	já existe um MySQL local	o projeto usa 3307; ou mude <code>MYSQL_HOST_PORT</code>
dotnet ef não encontrado	ferramenta local não restaurada	<code>dotnet tool restore</code> dentro de <code>backend/</code>
build falha com "arquivo em uso"	a API está rodando e trava as DLLs	pare a API (Ctrl+C) antes de compilar os testes
API não sobe: Unable to connect	MySQL parado	<code>docker compose up -d</code> e aguarde "healthy"
tela "Não foi possível carregar os dados"	API parada ou <code>API_URL</code> errada	suba a API; confira <code>.env.local</code>
testes de integração falham ao iniciar	Docker parado	abra o Docker Desktop
migration nova não aparece no banco	não foi aplicada	rode a API ou <code>dotnet ef database update</code>
quero recomeçar o banco do zero		<code>docker compose down -v</code> e suba de novo

24 Glossário

Termo	Significado
Action	método público de um controller que atende uma rota
Action filter	código que roda antes e/ou depois das actions (aqui: validação)
Agregado	grupo de objetos com uma raiz que garante as regras (Produto e suas movimentações)
Concorrência otimista	detectar, no <code>UPDATE</code> , que outro usuário alterou o registro antes (token no <code>WHERE</code>)
Controller	classe que agrupa as actions de um recurso HTTP
DbContext / DbSet	a sessão com o banco no EF Core / uma tabela dentro dela

Termo	Significado
DI (injeção de dependência)	o framework cria e entrega as dependências pelo construtor
DTO	objeto só de dados usado na fronteira (request e response)
Entidade / model de domínio	classe com identidade e regras, mapeada para uma tabela
Fluent API	configuração do mapeamento em código (<code>builder.Property(...)</code>)
Middleware	etapa do pipeline por onde toda requisição passa
Migration	classe gerada que evolui o schema do banco (<code>Up / Down</code>)
Model binding	conversão automática de rota, query e JSON em parâmetros da action
ORM	biblioteca que mapeia objetos para tabelas (EF Core)
ProblemDetails	formato padrão de erro HTTP em JSON (RFC 9457)
Result pattern	devolver sucesso ou erro como valor, em vez de lançar exceção
Server Action	função do Next.js com <code>"use server"</code> , chamada por formulários
Server / Client Component	componente React que roda só no servidor / também no navegador
Service	classe da Application que implementa os casos de uso
Snapshot	"foto" do modelo usada pelo EF para calcular a próxima migration

Para aprofundar, os guias em `docs/` no repositório detalham cada assunto, e `docs/05-exercicios.md` traz exercícios em ordem de dificuldade.